

PUC
RIO

Esse ai resolve com CCW

Thomaz Miranda, Miguel Batista, Joao Arthur Marques

November 3, 2026

Code Contents

1 Dynamic Programming 1
1.1 Divide and Conquer DP Optimization 1
1.2 Sum over Subsets DP (SOS DP) 1
2 Extra (Templates and Utilities) 1
2.1 template 1
2.2 Otimizacoes agressivas, pode deixar mais rapido ou mais devagar 1
2.3 random 1
2.4 makefile 1
2.5 hash 1
2.6 stress 1
2.7 tester 1
3 Computational Geometry 2
3.1 Circumcircle and Circumradius 2
3.2 Convex Hull (Monotone Chain) 2
3.3 Float Geometry Primitives 2
3.4 Integer Geometry Primitives 3
3.5 Halfplane Intersection 3
3.6 Point Inside Polygon 3
3.7 Minimum Enclosing Circle 3
3.8 Plota Poligonos 4
3.9 Polygon Cut 4
3.10 Segment Sweep Line Skeleton 4
4 Graph Algorithms 4
4.1 2-SAT 4
4.2 Articulation Points / Bridges / Biconnected Components 4
4.3 Centroid Decomposition - pares a distancia k 5
4.4 Centroid Tree 5
4.5 Dijkstra's Shortest Paths 5
4.6 Dinic's Maximum Flow (with Scaling) . . 5
4.7 Directed Minimum Spanning Tree (Edmonds / Chu-Liu) 6
4.8 Dominator Tree (Lengauer-Tarjan) 6
4.9 Floyd-Warshall Algorithm 6
4.10 Heavy Light Decomposition (Path query + Path update) 7
4.11 LCA com binary lifting 7
4.12 LCA com Euler tour + Sparse Table (RMQ) 7
4.13 Min Cost Max Flow (MCMF) 7
4.14 Strongly Connected Components (Kosaraju) 8
4.15 Topological Sort (Kahn's Algorithm) . . . 8
4.16 Virtual Tree 8
5 Mathematics 9
5.1 Chinese Remainder Theorem 9
5.2 Extended Euclidean Algorithm 9
5.3 NTT Convolution 9
5.4 Gauss - Z2 9
5.5 Gauss-Jordan 10
5.6 Matrix template 10
5.7 Miller-Rabin Primality Test (Deterministic for 64-bit) 11
5.8 Modular Integer 11
5.9 Modular Arithmetic Helpers 11
5.10 Discrete Logarithm (Baby-step Giant-step) . 11
5.11 Discrete Square Root 11
5.12 Primitive Root (Generator) 12
5.13 Quadratic Field Extension (Z_p[sqrt(D)]) . 12
5.14 Linear Sieve (prime precomputation) . . . 12
6 String Algorithms 13
6.1 Aho-Corasick Automaton 13
6.2 Knuth-Morris-Pratt (KMP) 13
6.3 Suffix Array - O(n log n) 13
6.4 Trie (Prefix Tree) 13
6.5 Z-function 14
7 Data Structures 14
7.1 Fenwick Tree with Range Updates 14
7.2 Convex Hull Trick Dinamico 14
7.3 DSU with Rollback 14
7.4 Disjoint Set Union (Union-Find) 14
7.5 Segment Tree Over Time (Offline Dynamic Connectivity) 15
7.6 Fenwick Tree (Binary Indexed Tree) 15
7.7 Custom Hash for hash table 15
7.8 Mo's Algorithm (Offline Range Queries) . 15
7.9 Order-Statistic Tree (PBDS) 16
7.10 Persistent Segment Tree 16
7.11 Segment Tree (Range Query + Range Update) 16
7.12 Segment Tree (Range Query + Point Update) 17
7.13 Sparse Table (Idempotent Range Query) . . 17
7.14 Implicit Treap (Sequence Treap) 17

Theoretical Contents

1 Counting Problems 18
1.1 Burnside's Lemma 18
1.2 Catalan Numbers 18
2 Graphs 18
2.1 Sum of Subtrees of a Tree 18
3 Number Theory 18
3.1 Chicken McNugget Theorem 18
3.2 Identities 18
3.3 Large Prime Gaps 18
3.4 Lucas's Theorem 19
3.5 Mobius Inversion 19
3.6 Number of Divisors 19
3.7 Prime Counting Function - $\pi(x)$ 19
3.8 Some Primes 19

1 Dynamic Programming

1.1 Divide and Conquer DP Optimization 186

Optimizes DP transitions with quadrangle inequality/monge-like structure using divide-and-conquer over optimal decision points.

Complexity: O(K N log N) with O(1) cost, O(N)

```
66b vector<vll> dp; // dp[n+1][2]

b8e void solve(ll k, ll l, ll r, ll lk, ll rk) {
de6     if (l > r) return;
f20     ll m = (l+r)/2, p = -1;
bff     auto& ans = dp[m][k&1] = INF;
f08     for (ll i = max(m, lk); i <= rk; i++) {
d73         ll at = dp[i+1][~k&1] + cost(m, i);
57d         if (at < ans) ans = at, p = i;
d63     }
1ee     solve(k, l, m-1, lk, p), solve(k, m+1, r, p, rk);
35b }

c5c ll dnc(ll n, ll k) {
321     dp[n][0] = dp[n][1] = 0;
390     forn(i,0,n) dp[i][0] = INF;
050     forn(i,1,k+1) solve(i, 0, n-i, 0, n-i);
8e7     return dp[0][k&1];
40f }
```

1.2 Sum over Subsets DP (SOS DP) 264

sos_dp_sub: for each mask, computes the sum of f[s] over all subsets s of mask (i.e., f'[mask] = sum of f[s] for s in mask). sos_dp_super: for each mask, computes the sum of f[s] over all supersets s of mask (i.e., f'[mask] = sum of f[s] for mask in s). input size must be a power of two (2^n).

Complexity: O(n * 2^n), O(2^n) mem

```
9e8 vll sos_dp_sub(vll f) {
f5d     ll n = __builtin_ctzll(sz(f));
310     assert((1ll<<n) == sz(f));

4cd     forn(i,0,n) forn(mask, 0, (1<<n))
796         if (mask>>i&1) f[mask] += f[mask^(1<<i)];
         // -= for inverse
abe     return f;
7c3 }

6b6 vll sos_dp_super(vll f) {
f5d     ll n = __builtin_ctzll(sz(f));
310     assert((1ll<<n) == sz(f));

4cd     forn(i,0,n) forn(mask, 0, (1<<n))
a3c         if (~mask>>i&1) f[mask] += f[mask^(1<<i)];
         // -= for inverse
abe     return f;
d18 }
```

2 Extra (Templates and Utilities)

2.1 template fa9

```
2b7 #include <bits/stdc++.h>
ca4 using namespace std;

ad1 typedef long long ll;
0d2 typedef pair<ll, ll> pll;
3a6 typedef vector<ll> vll;

4d1 #define forn(i, s, e) for (ll i = (s); i < (e); i++)
66d #define sz(u) ((ll) u.size())
cb9 #define ln "\n"

ba0 #ifdef DEBUG
2d9 #define trace(u) u
931 #define _
8c1 #else
56a #define trace(u)
aef #define _ ios::sync_with_stdio(0); cin.tie(0)
f2e #endif

721 #define debug(u) trace(cout << #u " = " << u << ln)
e98 #define debugv(v) trace(cout << #v ": "; for (auto xx : v) cout
<< xx << " "; cout << ln)

563 const ll INF = 0x3f3f3f3f3f3f3f3fll;
```

```
// int main() {
//     _;
//     return 0;
// }
```

2.2 Otimizacoes agressivas, pode deixar mais rapido ou mais devagar `35c`

```
#pragma GCC optimize("Ofast")
// Soft Ofast
#pragma GCC optimize("O3")
// Auto explicativo
#pragma GCC optimize("unroll-loops")
// Vetorizacao
#pragma GCC target("avx2")
// Para operacoes com bits
#pragma GCC target("bmi,bmi2,popcnt,lzcnt")
// tem que juntar os target na mesma linha
```

2.3 random `b0e`

```
mt19937_64 rng((ll) chrono::steady_clock::now().time_since_epoch().count());

ll uniform(ll l, ll r){
    uniform_int_distribution<ll> uid(l, r);
    return uid(rng);
}
```

2.4 makefile `885`

```
CXXFLAGS = -fsanitize=address,undefined -fno-omit-frame-pointer -g -Wall -Wshadow -std=c++20 -Wno-unused-result -Wno-sign-compare
```

2.5 hash `826`

```
sed -n $2',’$3’ p’ $1 | sed ’/^#w/d’ | cpp -dD -P -fpreprocessed | tr -d ’[:space:]’ | md5sum | cut -c-3
```

2.6 stress `d5d`

```
P=a
make ${P} ${P}2 gen || exit 1
for ((i = 1; ; i++)) do
    ./gen $i > tin
    ./${P} < tin > tout
    ./${P}2 < tin > tout2
    if (! cmp -s tout tout2) then
        echo "--> entrada:"
        cat tin
        echo "--> saida1:"
        cat tout
        echo "--> saida2:"
        cat tout2
        break;
    fi
    echo $i
done
```

2.7 tester `155`

```
P=a
make "$P" || exit 1

for i in {1..9}; do
    ./"$P" < "in/${P}${i}" > out_ &&
    cmp -s out_ "out/${P}${i}" &&
    echo "OK $i" ||
    { echo "NOT $i"; break; }
done
```

3 Computational Geometry

3.1 Circumcircle and Circumradius `c8f`

ccRadius: raio do circulo circunscrito. ccCenter: centro do circulo circunscrito. Pontos colineares -> retorna DINF / pt(DINF, DINF). Requer geo_double (pt de ld).

Complexity: O(1)

```
pt perp(pt p) { return pt(-p.y, p.x); } // rotaciona 90 graus anti-horario

ld ccRadius(pt a, pt b, pt c) {
    ld s = sarea(a, b, c);
    if (cmp(s) == 0) return DINF;
    return dist(a, b) * dist(b, c) * dist(c, a) / (4 * abs(s));
}

pt ccCenter(pt a, pt b, pt c) {
    b = b - a, c = c - a;
    if (cmp(b ^ c) == 0) return pt(DINF, DINF);
    return a + perp(b * norm_sq(c) - c * norm_sq(b)) / (2 * (b ^ c));
}
```

3.2 Convex Hull (Monotone Chain) `80e`

Computes the convex hull of a set of points using Andrew's monotone chain. Returns the hull in counterclockwise order, without repeated points. col = true keeps collinear points on the edges.

Complexity: O(N log N), O(N)

```
vector<pt> convex_hull(vector<pt> v, bool col = false) {
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());
    if (sz(v) <= 1) return v;
    vector<pt> l, u;
    forn(i, 0, sz(v)) {
        while (sz(l) > 1 && !ccw(l.end()[-2], l.end()[-1], v[i], !col))
            l.pop_back();
        l.push_back(v[i]);
    }
    if (sz(l) == sz(v)) return v; // todos na cadeia inferior ( inclui caso colinear)
    for (ll i = sz(v) - 1; i >= 0; i--) {
        while (sz(u) > 1 && !ccw(u.end()[-2], u.end()[-1], v[i], !col))
            u.pop_back();
        u.push_back(v[i]);
    }
    l.pop_back(); u.pop_back();
    for (pt p : u) l.push_back(p);
    return l;
}
```

3.3 Float Geometry Primitives `fca`

Defines 2D point and line structures with orientation, area, line intersection and circle-circle / circle-line intersections.

```
typedef long double ld;

const ld DINF = 1e18;
const ld eps = 1e-9;
const ld pi = 2*acos(0);

#define sq(x) ((x)*(x))

ll cmp(ld x, ld y = 0, ld tol = eps) {
    return (x <= y + tol) ? (x + tol < y) ? -1 : 0 : 1;
}

struct pt { // ponto
    ld x, y;
    pt(ld x_ = 0, ld y_ = 0) : x(x_), y(y_) {}
    bool operator < (const pt p) const {
        if (cmp(x, p.x) != 0) return cmp(x, p.x) < 0;
        if (cmp(y, p.y) != 0) return cmp(y, p.y) < 0;
        return 0;
    }
    bool operator == (const pt p) const {
        return cmp(x, p.x) == 0 && cmp(y, p.y) == 0;
    }

    pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
    pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
    pt operator * (const ld c) const { return pt(x*c  , y*c  ); }
    pt operator / (const ld c) const { return pt(x/c  , y/c  ); }
    ld operator * (const pt p) const { return x*p.x + y*p.y; }
    ld operator ^ (const pt p) const { return x*p.y - y*p.x; }
    friend istream& operator >> (istream& in, pt& p) {
        return in >> p.x >> p.y;
    }
};

struct line { // reta
    pt p, q;
    line() {}
    line(pt p_, pt q_) : p(p_), q(q_) {}
    friend istream& operator >> (istream& in, line& r) {
        return in >> r.p >> r.q;
    }
};

ld norm_sq(pt p){ return p*p;}

ld norm(pt p){ return sqrt(norm_sq(p)); }

ld dist_sq(pt p, pt q){ return norm_sq(p - q);}

ld dist(pt p,pt q){ return sqrt(dist_sq(p, q));}

ld angle(pt v) { // angulo do vetor com o eixo x
    ld ang = atan2(v.y, v.x);
    if (cmp(ang, 0) < 0) ang += 2*pi;
    return ang;
```

```
2e3 }

298 ld sarea(pt p, pt q, pt r) { // area com sinal
606     return ((q-p)^(r-q))/2;
1b1 }

e32 bool col(pt p, pt q, pt r) { // se p, q e r sao colin.
571     return cmp(sarea(p, q, r), 0) == 0;
e64 }

82b bool ccw(pt p, pt q, pt r, bool strict = true) { // se p, q, r
estao em sentido anti-horario
dc3     return cmp(sarea(p, q, r)) > 0 || (!strict && cmp(sarea(p, q,
 r)) == 0);
073 }

099 bool isinseg(pt p, line r) { // se p pertence ao seg de r
f65     pt a = r.p - p, b = r.q - p;
26e     return cmp(a ^ b) == 0 && cmp(a * b) <= 0;
fe1 }

4a6 ld polarea(vector<pt>& v) { // area do poligono
9c5     ld ret = 0;
ced     forn(i, 0, sz(v))
6d4         ret += sarea(pt(0, 0), v[i], v[(i + 1) % sz(v)]);
d03     return abs(ret);
d91 }

98d
ld get_t(pt v, line r) { // retorna t tal que t*v pertence a reta r
6ee     return (r.p^r.q) / ((r.p-r.q)^v);
a0a }

d5c pt inter(line r, line s) { // r inter s
9a6     if (cmp((r.p - r.q) ^ (s.p - s.q), 0) == 0) return pt(DINF,
DINF);
205     r.q = r.q - r.p, s.p = s.p - r.p, s.q = s.q - r.p;
543     return r.q * get_t(r.q, s) + r.p;
9be }

// Interseccao de circulos: pontos inexistentes = (DINF, DINF)
// concentricos (mesmo se iguais) -> nenhum ponto
551 pair<pt, pt> circint(pt o1, ld r1, pt o2, ld r2) {
24d     pt in(DINF, DINF), dir = o2 - o1;
837     ld d = norm(dir);
b3c     if (cmp(d) == 0) return {in, in};
95d     if (cmp(d, r1 + r2) > 0 || cmp(d, abs(r1 - r2)) < 0) return {
in, in};
48e     ld x = (d*d - r2*r2 + r1*r1) / (2*d);
a45     pt m = o1 + dir * (x / d);
3c8     if (cmp(d, r1 + r2) == 0 || cmp(d, abs(r1 - r2)) == 0)
return {m, in}; // tangentes
f6a
    pt k = pt(-dir.y, dir.x) * (sqrt(max((ld)0, r1*r1 - x*x)) / d);
4b1     return {m + k, m - k};
1e2 }

// Interseccao de circulo e reta

1fa pt foot(pt p, pt a, pt b) { // projecao de p na reta ab
a8a     pt dir = b - a;
dad     return (dir * ((p - a) * dir)) / (dir * dir) + a;
84d }

996 pair<pt, pt> cline(pt o, ld r, pt a, pt b) {
6b4     pt h = foot(o, a, b), in(DINF, DINF), dir = b - a;
897     ld d = norm(h - o);
feb     if (cmp(d, r) > 0) return {in, in};
b54     if (cmp(d, r) == 0) return {h, in}; // tangente
236     pt k = dir * (sqrt(r*r - d*d) / norm(dir));
194     return {h + k, h - k};
2f7 }
```

3.4 Integer Geometry Primitives 3e6

Defines 2D point and line structures with orientation, area, and angle comparisons plus a sweep-line comparator.

```
b2a struct pt { // point
0be     ll x, y;
f6f     pt(ll x_ = 0, ll y_ = 0) : x(x_), y(y_) {}
5bc     bool operator < (const pt p) const {
95a         if (x != p.x) return x < p.x;
89c         return y < p.y;
dcd     }
a83     bool operator == (const pt p) const {
48a         return x == p.x && y == p.y;
277     }
cb9
    pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
a24
    pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
8f0     pt operator * (const ll c) const { return pt(x*c, y*c); }
60d     ll operator * (const pt p) const { return x*(ll)p.x + y*(ll)
p.y; } // dot product
d86     ll operator ^ (const pt p) const { return x*(ll)p.y - y*(ll)
p.x; } // cross product
5ed     friend istream& operator >> (istream& in, pt& p) {
e37         return in >> p.x >> p.y;
e45     }
1b5 };

b3a struct line { // line segment
730     pt p, q;

0d6     line() {}
4b8     line(pt p_, pt q_) : p(p_), q(q_) {}
7f9     bool operator < (const line l) const {
d1d         if (!(p == l.p)) return p < l.p;
d4a         return q < l.q;
2ca     }
e1c     bool operator == (const line l) const {
05e         return p == l.p && q == l.q;
b27     }
8d7     friend istream& operator >> (istream& in, line& r) {
4cb         return in >> r.p >> r.q;
858     }
ab7 };

5a2 ll sarea2(pt p, pt q, pt r) { // 2 * signed area
586     return (q-p)^(r-q);
bf4 }

e58 ll norm_sq(pt p){ return p*p;}

baa ll dist_sq(pt p, pt q){ return norm_sq(p - q);}

82b bool ccw(pt p, pt q, pt r, bool strict = true) {
966
    return sarea2(p, q, r) > 0 || (!strict && sarea2(p, q, r) == 0);
40e }

c31 int quad(pt p) { // quadrant of a point
dbb     return (p.x<0)^3*(p.y<0);
fcf }

2df bool compare_angle(pt p, pt q) { // returns true if angle(p) <
angle(q)
9fc     if (quad(p) != quad(q)) return quad(p) < quad(q);
ea1     return ccw(q, pt(0, 0), p);
771 }

099 bool isinseg(pt p, line r) { // se p pertence ao seg de r
f65     pt a = r.p - p, b = r.q - p;
3e1     return (a ^ b) == 0 && (a * b) <= 0;
562 }

676 bool interseg(line r, line s) { // se o seg de r intersecta o
seg de s
739     if (isinseg(r.p, s) || isinseg(r.q, s)
bdc         || isinseg(s.p, r) || isinseg(s.q, r)) return 1;

13c     return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) &&
413             ccw(s.p, s.q, r.p) != ccw(s.p, s.q, r.q);
d79 }

510 ll polarea(vector<pt> v) { // area do poligono
b73     ll ret = 0;
ced     forn(i, 0, sz(v))
382         ret += sarea2(pt(0, 0), v[i], v[(i + 1) % sz(v)]);
d03     return abs(ret);
76b }

// comparator for set to perform sweep line with segments
2c4 struct cmp_sweepline {
d80     bool operator () (const line& a, const line& b) const {
         // assumes segments have p < q
191         if (a.p == b.p) return ccw(a.p, a.q, b.q);
b4b         if (a.p.x != a.q.x && (b.p.x == b.q.x || a.p.x < b.p.x))
780             return ccw(a.p, a.q, b.p);
dc0         return ccw(a.p, b.q, b.p);
187     }
21b };
```

3.5 Halfplane Intersection 667

Acha o ponto mais extremo da intersecao na direcao best_dir. So entram retas nao verticais: processe as verticais antes e codifique-as em minx e maxx. Para o ponto mais acima, use best_dir = (0, 1); conforme best_dir, best = (+-INF, +-INF).

```
c79 mt19937_64 rng((ll) chrono::steady_clock::now().time_since_epoch
().count());

877 pt hpi_1d(vector<line> v, pt best_dir, ll n, line suport, ll
minx, ll maxx){
3ec     pt low = inter(line(pt(minx, 0), pt(minx,1)), suport), upp =
 inter(line(pt(maxx, 0), pt(maxx,1)), suport);
522     forn(i, 0, n){
beb         pt intersection = inter(v[i], suport);
f1c         debug(low);
dda         debug(upp);
```

```
f9d         debug(intersection);
138         debug(v[i]);

5b8         if(!ccw(v[i].p, v[i].q, low)) low = intersection;
030         else if(!ccw(v[i].p, v[i].q, upp)) upp = intersection;
9b8     }

a7b     if(cmp(low.x, upp.x) > 0) return {DINF, DINF};

5b1     if(cmp(best_dir*low, best_dir*upp) <= 0) return upp;
b2b     return low;
b43 }

219 pt hpi(vector<line> v, pt best_dir, ll minx, ll maxx){
f21     shuffle(v.begin(), v.end(), rng);

dce     ll n = sz(v);
3a3     pt best(1, 1);

522     forn(i, 0, n){
20a         debug(best);
138         debug(v[i]);
121         if(ccw(v[i].p, v[i].q, best)) continue;
186         best = hpi_1d(v, best_dir, i, v[i], minx, maxx);
ff8         assert(!(best ==pt(DINF, DINF)));
4c5     }

f26     return best;
f0a }
```

3.6 Point Inside Polygon 11a

Diz se um ponto esta dentro do poligono (ray casting). Com strict = true, pontos na borda nao contam como interior.

Complexity: O(n)

```
ffe #include "src/geometry/geo_double.cpp"

f40 bool inpolygon(vector<pt> &p, pt a, bool strict = true) {
974     ll cnt = 0, n = sz(p);
522     forn(i,0,n) {
61c         pt q = p[(i + 1) % n];
03a         if (isinseg(a, line(p[i], q))) return !strict;
793         cnt ^= (((cmp(a.y, p[i].y) < 0) - (cmp(a.y,q.y) < 0))*((
p[i]-a)^(q-a))) > 0 ;
dc9     }
70a     return cnt;
a17 }
```

3.7 Minimum Enclosing Circle b3c

*Menor circulo que contem todos os pontos. Aleatorizado. So funciona com pt de double; talvez usar r * (1 + eps) na comparacao.*

Complexity: O(n) esperado

```
faf pair<pt, ld> mec(vector<pt> v) {
7c9     shuffle(v.begin(), v.end(), mt19937(time(0)));
d91     pt o = v[0];
e1e     ld r = 0;
eb8     forn(i, 0, sz(v)) if (cmp(dist(o, v[i]),r) > 0) {
983         o = v[i], r = 0;
141         forn(j, 0, i) if (cmp(dist(o, v[j]),r) > 0) {
76e             o = (v[i] + v[j]) / 2;
f66             r = dist(o, v[i]);
21c             forn(k, 0, j) if (cmp(dist(o, v[k]),r) > 0) {
6b1                 o = ccCenter(v[i], v[j], v[k]);
f66                 r = dist(o, v[i]);
8d3             }
c74         }
200     }
645     return {o, r};
b3c }
```

3.8 Plota Poligonos 73d

Ajuda a debugar codigo de geometria

```
def svg(P, o="d.svg"):
    if not P: return
    X, Y = zip(*P)
    x0, x1, y0, y1 = min(X), max(X), min(Y), max(Y)
    p = max(x1-x0, y1-y0) * 0.1 or 10
    head = f'<svg viewBox="{x0-p} {y0-p} {x1-x0+2*p} {y1-y0+2*p}"
xmlns="http://www.w3.org/2000/svg" style="background:#eee">'
    poly = f'<polygon points="{" ".join(f"{x},{y}" for x,y in P)}"
fill="#f003" stroke="red" stroke-width="{p/20}"/>'
    text = "".join([f'<text x="{x}" y="{y}" font-size="{p/3}">{i}</
text>' for i,(x,y) in enumerate(P)])
    with open(o, 'w') as f: f.write(head + poly + text + '</svg>')
```

3.9 Polygon Cut 00d

Corta o poligono pela reta l, mantendo so a parte ccw em relacao a l.

Complexity: O(n)

```
39e vector<pt> polygonCut(const vector<pt>& poly, line l) {
819     vector<pt> res;
e29     forn(i,0,sz(poly)) {
07c         pt curr = poly[i], prev = i ? poly[i-1] : poly.back();
dc8         bool a = ccw(l.p, l.q, curr), b = ccw(l.p, l.q, prev);
123         if (a != b)
1e1             res.push_back(inter(line(curr, prev), l));
754         if (a)
fb3             res.push_back(curr);
c0d     }
b50     return res;
00d }
```

3.10 Segment Sweep Line Skeleton def

Maintains an active set of segments ordered for sweep-line processing over x; insertion and removal are typically logarithmic.

```
719 map<ll, set<line, cmp_sweepline>> sweepline_begin; // dado um x,
 diz quais linhas comecam naquele x
5a6 map<ll, set<line, cmp_sweepline>> sweepline_end;   // dado um x,
 diz quais linhas terminam naquele x

765 void process_beg(set<line, cmp_sweepline>& v, set<line,
cmp_sweepline>& active_line, vll& parent){
47d     for(auto x : v){
380         active_line.insert(x);
         // processar uma linha que esta sendo adicionada
9c5     }
fb4 }

923 void process_end(set<line, cmp_sweepline>& v, set<line,
cmp_sweepline>& active_line){
47d     for(auto x : v){
d76         active_line.erase(x);
a1d     }
68a }

ec5 void sweepline(ll n){
23e     set<line, cmp_sweepline> active_line;

521     while(!sweepline_begin.empty() || !sweepline_end.empty()){
c58         auto it_beg = sweepline_begin.begin();
aa0         auto it_end = sweepline_end.begin();

385         if(sweepline_end.empty()){
570             process_beg(it_beg->second, active_line, parent);
7ae             sweepline_begin.erase(it_beg);
5e2             continue;
a8b         }

379         if(sweepline_begin.empty() || it_end->first <= it_beg->
first){
2a4             process_end(it_end->second, active_line);
61a             sweepline_end.erase(it_end);
5e2             continue;
ef4         }

570         process_beg(it_beg->second, active_line, parent);
7ae         sweepline_begin.erase(it_beg);
b06     }
76d }
```

4 Graph Algorithms

4.1 2-SAT 349

solve() retorna um par: first diz se existe atribuicao valida, second[i] diz se a variavel i e verdadeira.

Complexity: O(#variaveis + #restricoes)

```
138 struct sat {
446     ll n, tot;
838     vector<vll> g;
9ba     vll vis, comp, id, ans;
d1b     vll s;

141     sat() {}
5f5     sat(ll n_) : n(n_), tot(n), g(2*n) {}

81c     ll dfs(ll i, ll& t) {
02f         ll lo = id[i] = t++;
c3f         s.push_back(i), vis[i] = 2;
9c7         for (ll j : g[i]) {
740             if (!vis[j]) lo = min(lo, dfs(j, t));
994             else if (vis[j] == 2) lo = min(lo, id[j]);
3aa         }
3de         if (lo == id[i]) while (1) {
b8e             ll u = s.back(); s.pop_back();
9c5             vis[u] = 1, comp[u] = i;
121             if ((u>>1) < n && ans[u>>1] == -1) ans[u>>1] = ~u&1;
2ef             if (u == i) break;
9fd         }
253         return lo;
c25     }

286     void add_impl(ll x, ll y) { // x -> y = !x ou y
26a         x = x >= 0 ? 2*x : -2*x-1;
2b8         y = y >= 0 ? 2*y : -2*y-1;
```

```
a1e         g[x].push_back(y);
1e2         g[y^1].push_back(x^1);
677     }
9b5     void add_cl(ll x, ll y) { // x ou y
0b5         add_impl(~x, y);
135     }
37f     void add_xor(ll x, ll y) { // x xor y
0b7         add_cl(x, y), add_cl(~x, ~y);
4ac     }
680     void add_eq(ll x, ll y) { // x = y
c86         add_xor(~x, y);
9b1     }
874     void add_true(ll x) { // x = T
18b         add_impl(~x, x);
ea8     }
44e     void at_most_one(vll v) { // no max um verdadeiro
051         g.resize(2*(tot+sz(v)));
6fe         forn(i, 0, sz(v)) {
8c9             add_impl(tot+i, ~v[i]);
a8f             if (i) {
b6a                 add_impl(tot+i, tot+i-1);
3d3                 add_impl(v[i], tot+i-1);
0f7             }
472         }
253         tot += sz(v);
0a2     }

806     pair<bool, vll> solve() {
c79         ans = vll(n, -1);
b1b         ll t = 0;
449         vis = comp = id = vll(2*tot, 0);
f58         forn(i, 0, 2*tot) if (!vis[i]) dfs(i, t);
286         forn(i, 0, tot)
4c9             if (comp[2*i] == comp[2*i+1]) return {false, {}};
997         return {true, ans};
9a9     }
349 };
```

4.2 Articulation Points / Bridges / Biconnected Components a7c

is_art[i]: i e ponto de articulacao (ART). bridges: lista de arestas ponte (BRIDGE). bcc: lista de componentes biconexas, em arestas (BCC).

Complexity: O(n + m)

```
f08 struct TJ {
26a     ll n, timer=0;
cb0     vector<vll>& g;
fc9     vll tin, low;

f15     vector<bool> is_art; // ART
5f2     vector<pll> bridges; // BRIDGE
950     vector<pll> st; // BCC
f5e     vector<vector<pll>> bcc; // BCC

1fc     TJ(ll n, vector<vll>& g): n(n), g(g){
cfd         tin.assign(n,-1);
af8         low.resize(n);
1a1         is_art.assign(n, false); // ART
4bc         forn(i, 0, n) if(tin[i]==-1) dfs(i);
56c     }

b6d     void dfs(ll v, ll p=-1){
f3d         tin[v]=low[v]=timer++;
4fc         ll ch=0; bool pe=false;

72c         for(ll to:g[v]){
a2e             if(to==p && !pe){ pe=true; continue; }

a5f             if(tin[to]==-1){
df6                 st.push_back({v,to}); // BCC

a72                 dfs(to,v);
575                 low[v] = min(low[v], low[to]);

f03                 if(low[to]>tin[v]) bridges.push_back({v,to}); //
 BRIDGE
88a                 if(low[to]>=tin[v] && p!=-1) is_art[v] = true; //
 ART

6aa                 if(low[to]>=tin[v]){ // BCC
cc9                   vector<pll> c; // BCC
31e                   while(1){ // BCC
69d                     auto e=st.back(); st.pop_back(); // BCC
d5e                     c.push_back(e); // BCC
ffa                     if(e.first==v && e.second==to) break; // BCC
cd6                   } // BCC
fc8                   bcc.push_back(c); // BCC
155                 } // BCC

dfc                 ch++;
0a9             }else{
6de                 low[v]=min(low[v],tin[to]);
328
                }
498             if(tin[to]<tin[v]) st.push_back({v,to});   // BCC
34e         }
30a         if(p==-1 && ch>1) is_art[v] = true;   // ART
c83     }
a7c };
```

4.3 Centroid Decomposition - pares a distancia k 9a2

Conta os pares (u, v) com dist(u, v) = k na arvore. Em cada centroide, para cada subarvore, soma cnt[k-d-1] antes de inserir as profundidades d dela, para nao contar o mesmo par duas vezes. rem e restaurado no final, entao da para chamar de novo com outro k.

Complexity: O(n log n), O(n)

```
1fd const ll MAX = 100'005;
32a vll g[MAX];
a49 ll siz[MAX], rem[MAX];

945 void dfs(vll& path, ll i, ll l=-1, ll d=0) {
547     path.push_back(d);
113
    for (ll j : g[i]) if (j != l && !rem[j]) dfs(path, j, i, d+1);
3ee }

499 ll dfs_sz(ll i, ll l=-1) {
ae3     siz[i] = 1;
cac
    for (ll j : g[i]) if (j != l && !rem[j]) siz[i] += dfs_sz(j, i);
c61     return siz[i];
79e }

c46 ll centroid(ll i, ll l, ll size) {
9fe
    for (ll j : g[i]) if (j != l && !rem[j] && siz[j] > size / 2)
735         return centroid(j, i, size);
d9a     return i;
d5e }

27a ll decomp(ll i, ll k) {
79c     ll c = centroid(i, i, dfs_sz(i));
a67     rem[c] = 1;

     // gasta O(n) aqui - dfs sem ir pros caras removidos
04b     ll ans = 0;
8e1     vll cnt(siz[i]);
878     cnt[0] = 1;
e65     for (ll j : g[c]) if (!rem[j]) {
a0e         vll path;
baf         dfs(path, j);
ed7         for (ll d : path) if (0 <= k-d-1 && k-d-1 < siz[i])
285             ans += cnt[k-d-1];
477         for (ll d : path) cnt[d+1]++;
035     }

ffb     for (ll j : g[c]) if (!rem[j]) ans += decomp(j, k);
3f1     rem[c] = 0;
ba7     return ans;
de1 }
```

4.4 Centroid Tree a22

p[i]: pai de i na centroid tree. dist[i][k]: distancia na arvore original entre i e seu k-esimo ancestral na centroid tree.

Complexity: O(n log n) de tempo e memoria

```
1fd const ll MAX = 100'005;
70d vector<vll> g(MAX), dist(MAX);
d2b vll siz(MAX), rem(MAX), p(MAX);

499 ll dfs_sz(ll i, ll l=-1) {
ae3     siz[i] = 1;
cac
    for (ll j : g[i]) if (j != l && !rem[j]) siz[i] += dfs_sz(j, i);
c61     return siz[i];
79e }

c46 ll centroid(ll i, ll l, ll size) {
9fe
    for (ll j : g[i]) if (j != l && !rem[j] && siz[j] > size / 2)
735         return centroid(j, i, size);
d9a     return i;
d5e }

3de void dfs_dist(ll i, ll l, ll d=0) {
541     dist[i].push_back(d);
1b0     for (ll j : g[i]) if (j != l && !rem[j])
82a         dfs_dist(j, i, d+1);
225 }

457 void decomp(ll i, ll l = -1) {
79c     ll c = centroid(i, i, dfs_sz(i));
1b9     rem[c] = 1, p[c] = l;
534     dfs_dist(c, c);
1ef     for (ll j : g[c]) if (!rem[j]) decomp(j, c);
f75 }

145 void build(ll n) {
b26     forn(i,0,n) rem[i] = 0, dist[i].clear();
```

```
867     decomp(0);
40c     forn(i,0,n) reverse(dist[i].begin(), dist[i].end());
9d9 }
```

4.5 Dijkstra's Shortest Paths 699

Computes single-source shortest paths on non-negative weighted graphs using a priority queue.

Complexity: O((N + M) log N), O(N + M)

```
965 void dijkstra(ll s, vll &d, vll& p, vector<vector<pll>>& g) {
5b8     ll n = sz(g);
355     d.assign(n, INF);
d8d     p.assign(n, -1);

d66     d[s] = 0;
924     priority_queue<pll> pq;
7ba     pq.push({0, s});
502     while (!pq.empty()) {
5cd         ll u = pq.top().second;
6fd         ll d_u = -pq.top().first;
716         pq.pop();

211         if (d_u != d[u]) continue;

9c3         for (auto [v, w_v] : g[u]) {
f35             if (d[u] + w_v < d[v]) {
7ca                 d[v] = d[u] + w_v;
e42                 p[v] = u;
2a6                 pq.push({-d[v], v});
e72             }
48a         }
e88     }
699 }
```

4.6 Dinic's Maximum Flow (with Scaling) b72

Computes max flow using Dinic's algorithm with optional capacity scaling to speed up BFS levels. add(a, b, c): aresta direcionada a -> b com capacidade c (nao-direcionada: add(a, b, c) e add(b, a, c)) max_flow acumula em F; chamar de novo continua de onde parou, use reset() para zerar. min_cut() so e valido depois de max_flow (usa lev da ultima BFS); retorna as arestas originais do lado de s para o lado de t. Scaling comeca em 2^30: com capacidades maiores continua correto, mas perde eficiencia.

Complexity: O(V^2 E) geral, O(V E log U) com scaling, O(E sqrt V) em grafo unitario; O(V + E) memoria

```
472 struct dinitz {
f61     const bool scaling;
d74     ll lim;
670     struct edge {
283         ll to, cap, rev, flow;
7f9         bool res;
764         edge(ll to_, ll cap_, ll rev_, bool res_)
a94
            : to(to_), cap(cap_), rev(rev_), flow(0), res(res_) {}
eb4     };

002     vector<vector<edge>> g;
697     vll lev, beg;
a71     ll F;
9da
    dinitz(ll n, bool scaling_) : scaling(scaling_), g(n), F(0){}
f3e     void add(ll a, ll b, ll c) {
de2         g[a].emplace_back(b, c, sz(g[b]), false);
a1d         g[b].emplace_back(a, 0, sz(g[a])-1, true);
3d3     }

6d8     bool bfs(ll s, ll t) {
f09         lev = vll(sz(g), -1); lev[s] = 0;
abd         beg = vll(sz(g), 0);
7a6         queue<ll> q; q.push(s);
647         while (sz(q)) {
c79             ll u = q.front(); q.pop();
bd9             for (auto& i : g[u]) {
e4f
                if (lev[i.to] != -1 || (i.flow == i.cap)) continue;
17e                 if (scaling && i.cap - i.flow < lim) continue;
185                 lev[i.to] = lev[u] + 1;
8ca                 q.push(i.to);
00d             }
823         }
0de         return lev[t] != -1;
d7c     }

b95     ll dfs(ll v, ll t, ll f = INF) {
3e6         if (!f || v == t) return f;
78b         for (ll& i = beg[v]; i < sz(g[v]); i++) {
027             auto& e = g[v][i];
206             if (lev[e.to] != lev[v] + 1) continue;
934             ll foi = dfs(e.to, t, min(f, e.cap - e.flow));
749             if (!foi) continue;
3c5             e.flow += foi, g[e.to][e.rev].flow -= foi;
45c             return foi;
862         }
bb3         return 0;
468     }

074     ll max_flow(ll s, ll t) {
997         for (lim = scaling ? (1ll<<30) : 1; lim; lim /= 2)
69c             while (bfs(s, t)) while (ll ff = dfs(s, t)) F += ff;
4ff         return F;
a5d     }

f47     vector<pll> min_cut() {
956         vector<pll> cut;
21f         forn(u, 0, sz(g))
162             if (lev[u] != -1)
d4c                 for (auto& e : g[u])
554                     if (lev[e.to] == -1 && !e.res)
13d                         cut.push_back({u, e.to});
d1b         return cut;
667     }

e30     void reset() {
59f         F = 0;
843         for (auto& edges : g) for (auto& e : edges) e.flow = 0;
5d0     }
b72 };
```

4.7 Directed Minimum Spanning Tree (Edmonds / Chu-Liu) 849

finds the minimum cost arborescence rooted at src in a weighted directed graph. returns {cost, parent} where parent[u] is the predecessor of u in the optimal tree. returns {-1, {}} if no arborescence exists (with negative weights, check par.empty() instead of cost == -1).

Complexity: O(M log N), O(N+M) memory

```
43b #include "src/structures/dsu_rb.cpp"

d44 struct Edge {ll a, b, w; };
bf2 struct Node {
25f     Edge key;
c17     Node *l, *r;
981     ll delta;
a9c     void prop() {
6f9         key.w += delta;
d2d         if (l) l->delta += delta;
d86         if (r) r->delta += delta;
978         delta = 0;
0d3     }
866     Edge top() { prop(); return key; }
ab4 };
3eb Node *merge(Node *a, Node *b) {
b9f     if (!a || !b) return a ?: b;
626     a->prop(), b->prop();
dc2     if (a->key.w > b->key.w) swap(a, b);
485     swap(a->l, (a->r = merge(b, a->r)));
3f5     return a;
c51 }
7bb void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

408 pair<ll, vll> dmst(ll n, ll src, vector<Edge>& g) {
501     dsu_rb uf(n);
3f8     vector<Node*> heap(n);
5b2     vector<Node> pool(sz(g)); // avoids new (no memory leak)
637     forn(i,0,sz(g)) pool[i] = {g[i]}, heap[g[i].b] = merge(heap[
g[i].b], &pool[i]);
cd2     ll res = 0;
b15     vll seen(n, -1), path(n), par(n);
169     seen[src] = src;
dd6     vector<Edge> Q(n), in(n, {-1,-1}), comp;
71b     deque<tuple<ll, ll, vector<Edge>>> cycs;
f5b     forn(s,0,n) {
02e         ll u = s, qi = 0, w;
a0a         while (seen[u] < 0) {
572             if (!heap[u]) return {-1,{}};
ebe             Edge e = heap[u]->top();
5ed             heap[u]->delta -= e.w, pop(heap[u]);
952             Q[qi] = e, path[qi++] = u, seen[u] = s;
d56             res += e.w, u = uf.find(e.a);
9e2             if (seen[u] == s) { /// found cycle, contract
28d                 Node* cyc = 0;
ab5                 ll end = qi, time = uf.time();
f38                 do cyc = merge(cyc, heap[w = path[--qi]]);
4ec                 while (uf.unite(u, w));
562                 u = uf.find(u), heap[u] = cyc, seen[u] = -1;
c06                 cycs.push_front({u, time, {&Q[qi], &Q[end]}});
417             }
26e         }
f23         forn(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
376     }

973     for (auto& [u,t,com] : cycs) { // restore sol (optional)
36c         uf.rollback(t);
1d0         Edge inEdge = in[u];
854         for (auto& e : com) in[uf.find(e.b)] = e;
56d         in[uf.find(inEdge.b)] = inEdge;
ac5     }
```

```
271     forn(i,0,n) par[i] = in[i].a;
efb     return {res, par};
050 }
```

4.8 Dominator Tree (Lengauer-Tarjan) 583

builds the dominator tree given a directed graph and a source. returns a vector where ret[u] is the immediate dominator of u. ret[src] = -1 and ret[u] = -1 for u unreachable from src. Recursive DFS: a path-like graph with N ~ 2e5 may overflow a small stack.

Complexity: O((N+M) log N), O(N+M) mem

```
047 vll DomTree(vector<vll>& graph, ll src) {
9cf     ll n = sz(graph);
bab     vector<vll> tree(n), trans(n), buck(n);
2ea
    vll semi(n), par(n), dom(n), label(n), atob(n, -1), btoa(n, -1),
095         link(n, -1);

418     function<ll(ll, ll)> find=[&](ll u, ll d){
3d4         if(link[u] == -1) return d ? -1 : u;
9ff         ll v = find(link[u], d + 1);
388         if(v < 0) return u;
54b         if(semi[label[link[u]]] < semi[label[u]]) label[u] =
label[link[u]];
510         link[u] = v;
f95         return d ? v : label[u];
c83     };
b1b     ll t = 0;
5c2     function<void(ll)> dfs=[&](ll u){
640         atob[u] = t;
304         btoa[t] = u;
119         label[t] = semi[t] = t;
c82         t++;
612         for(auto v : graph[u]){
4f1         if(atob[v] == -1) dfs(v), par[atob[v]] = atob[u];
8b2         trans[atob[v]].push_back(atob[u]);
46c         }
155     };
088     dfs(src);
d48     for(ll u = t - 1; u >= 0; --u){
104         for(auto v : trans[u]) semi[u] = min(semi[u], semi[find(
v, 0)]);
9cf         if(u) buck[semi[u]].push_back(u);
9d5         for(auto w : buck[u]){
5d8             ll v = find(w, 0);
1db             dom[w] = semi[v] == semi[w] ? semi[w] : v;
c92         }
894         if(u) link[u] = par[u];
54f     }
ad9     vll ret(n, -1);
89d     forn(u,1,t) {
9c5         if(dom[u] != semi[u]) dom[u] = dom[dom[u]];
1eb         ret[btoa[u]] = btoa[dom[u]];
790     }
edf     return ret;
583 }
```

4.9 Floyd-Warshall Algorithm e53

Computes all-pairs shortest paths via DP over intermediate vertices. Initialize d[i][j] = INF, d[i][i] = 0, then d[u][v] = min(d[u][v], w) for each edge (handles multi-edges and negative self-loops). Returns true if a negative cycle is detected. Optional pass at the end sets d[i][j] = -INF for pairs whose shortest path is unbounded (passes through a negative cycle).

Complexity: O(N^3), O(N^2)

```
020 bool floyd_warshall(ll n, vector<vll>& d) {
b63     forn(k,0,n)
ded     forn(i,0,n)
b4c     forn(j,0,n)
bf8         if (d[i][k] < INF && d[k][j] < INF)
a2f
            d[i][j] = max(-INF, min(d[i][j], d[i][k] + d[k][j]));

599     bool neg = false;
ded     forn(i,0,n)
26f         if (d[i][i] < 0) neg = true;

    // optional: mark pairs affected by negative cycles
575     if (neg)
b13         forn(k,0,n) if (d[k][k] < 0)
637         forn(i,0,n) if (d[i][k] < INF)
3d7         forn(j,0,n) if (d[k][j] < INF)
724         d[i][j] = -INF;

a90     return neg;
e53 }
```

4.10 Heavy Light Decomposition (Path query + Path update) 9b9

Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most log(n) light edges. All values initialized to the segtree default. Root must be 0. adj must be undirected (u in adj[v] and v in adj[u]). With the sum Node, call set_leaves with cnt = 1 (otherwise updates do nothing). Recursive DFS: a line tree with N ~ 5e5 may overflow a small stack. head[u]: Highest node (closest to root) of the heavy path containing u. pos[u]: Position of node u in the Segment Tree array (DFS order). Uses a Lazy Segment Tree

Complexity: O((log N)^2) per op, O(N) build

```
e8f #include "src/structures/segtree_lazy.cpp"

303 template <bool VALS_EDGES> struct HLD {
6fa     ll N, tim = 0;
059     vector<vll> adj;
65c     vll parent, siz, head, pos;
9a7     std::unique_ptr<segtree<Node, Update>> seg;
ffd     HLD(vector<vll> adj_)
d3e         : N(sz(adj_)), adj(adj_), parent(N, -1), siz(N, 1),
a3f           head(N),pos(N){ dfsSz(0); dfsHld(0);
daa             seg = make_unique<segtree<Node, Update>>(N);
0e4         }
987     void set_leaves(const vector<Node>& lvs) {
         // if VALS_EDGES lvs[i] is the weight of i -> parent[i]
384         vector<Node> leaves(N);
1a0         forn(i, 0, N) leaves[pos[i]] = lvs[i];
61b         seg->set_leaves(leaves);
a26     }
eb1     void dfsSz(ll v) { // get heavy son
d10         for (ll& u : adj[v]) {
44a             adj[u].erase(find(adj[u].begin(), adj[u].end(), v));
308             parent[u] = v;
a00             dfsSz(u);
a02             siz[v] += siz[u];
68d             if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
4af         }
774     }
7f7     void dfsHld(ll v) { // "linearizes" the tree
06c         pos[v] = tim++;
1c7         for (ll u : adj[v]) {
c0d             head[u] = (u == adj[v][0] ? head[v] : u);
9f9             dfsHld(u);
9af         }
e38     }
aa3     template <class B> void process(ll u, ll v, B op) {
2e7         for (;; v = parent[head[v]]) {
df0             if (pos[head[u]] > pos[head[v]]) swap(u, v);
a57             if (head[u] == head[v]) break;
ad9             op(pos[head[v]], pos[v]);
a34         }
bdc         if (pos[u] > pos[v]) swap(u, v);
adf         op(pos[u] + VALS_EDGES, pos[v]);
d5d     }
050     void modifyPath(ll u, ll v, Update upd) {
3d8         process(u, v, [&](ll l, ll r) {
576             seg->update(l, r, upd);
ebf         });
ce2     }
d0b     Node queryPath(ll u, ll v) {
0f1         Node res = Node();
         // Non-commutative ops require exact merge order.
3d8         process(u, v, [&](ll l, ll r) {
f23             res = res*seg->query(l, r);
9b9         });
b50         return res;
584     }
bb3     Node querySubtree(ll v) {
012
        return seg->query(pos[v] + VALS_EDGES, pos[v] + siz[v] - 1);
0f7     }
142     void modifySubtree(ll v, Update upd) {
e4c
        seg->update(pos[v] + VALS_EDGES, pos[v] + siz[v] - 1, upd);
aa8     }
6d1 };
```

4.11 LCA com binary lifting b5e

Constroi com o grafo (lista de adjacencia) e a raiz. Um vertice e ancestral dele mesmo: se a e ancestral de b, lca(a, b) = a.

Complexity: build O(n log n), lca O(log n)

```
4d7 struct LCA_BL {
4ec     ll n, LOG, p = 0;
0e1     vector<vll> parent;
ed7     vll in, out;

bb6     LCA_BL(vector<vll>& g, ll raiz) : n(sz(g)), LOG(__lg(max(n,
1ll)) + 1),
02c         parent(LOG, vll(n)), in(n, -1), out(n) {
4c7         forn(i,0,n) parent[0][i] = i;
79b         dfs(g, raiz);
355         forn(k,1,LOG) forn(i,0,n)
ed8             parent[k][i] = parent[k - 1][parent[k - 1][i]];
b3f     }

00e     void dfs(vector<vll>& g, ll k) {
fdf         in[k] = p++;
2e0         for(ll v: g[k]){
e58             if(in[v] == -1){
5ad                 parent[0][v] = k;
1a5                 dfs(g, v);
```

```
            }
        }
        out[k] = p++;
    }

    bool anc(ll a, ll b) { // se a eh ancestral de b
        return in[a] <= in[b] && out[a] >= out[b];
    }

    ll lca(ll a, ll b) {
        if (anc(a, b)) return a;
        if (anc(b, a)) return b;
        for (ll k = LOG - 1; k >= 0; k--)
            if (!anc(parent[k][a], b)) a = parent[k][a];
        return parent[0][a];
    }
};
```

4.12 LCA com Euler tour + Sparse Table (RMQ) d01

Constroi com o grafo (lista de adjacencia) e a raiz. DFS recursiva: arvore-linha com N ~ 5e5 pode estourar pilha pequena.

Complexity: build O(n log n), lca e dist O(1)

```
#include "src/structures/sparse_table.cpp"

struct LCA {
    ll T = 0;
    vll time, path, ret, dep;
    sparse rmq;

    LCA(vector<vll>& C, ll root) : time(sz(C)), dep(sz(C)), rmq
((dfs(C,root,-1), ret)) {}
    void dfs(vector<vll>& C, ll v, ll par) {
        time[v] = T++;
        if(par != -1) dep[v] = dep[par] + 1;
        for (ll y : C[v]) if (y != par) {
            path.push_back(v), ret.push_back(time[v]);
            dfs(C, y, v);
        }
    }

    ll lca(ll a, ll b) {
        if (a == b) return a;
        tie(a, b) = minmax(time[a], time[b]);
        return path[rmq.query(a, b-1)];
    }

   ll dist(ll a,ll b){return dep[a] + dep[b] - 2*dep[lca(a,b)];}
};
```

4.13 Min Cost Max Flow (MCMF) c04

min_cost_flow(s, t, f) computes the pair (flow, cost) where max(flow) <= f with min(cost). min_cost_flow(s, t) -> Maximum flow with minimum cost from s to t. For DAGs, SPFA can be replaced with DP to avoid O(nm) initial cost. If no negative cost edges exist, SPFA is not needed. Nao suporta ciclos de custo negativo (SPFA entra em loop). Chamar de novo continua a partir do fluxo atual (nao zera).

Complexity: O(nm + f * m log n), O(n + m)

```
template<typename T> struct mcmf {
    struct edge {
        ll to, rev, flow, cap; // destination, reverse edge id,
current flow, capacity
        bool res; // is reverse edge
        T cost; // cost per unit of flow
        edge() : to(0), rev(0), flow(0), cap(0), res(false),
cost(0) {}

        edge(ll to_, ll rev_, ll flow_, ll cap_, T cost_, bool res_)
            : to(to_), rev(rev_), flow(flow_), cap(cap_), res(
res_), cost(cost_) {}
    };

    vector<vector<edge>> g;
    vll par_idx, par;
    T inf;
    vector<T> dist;

    mcmf(ll n) : g(n), par_idx(n), par(n), inf(numeric_limits<T
>::max()/3) {}

    void add(ll u, ll v, ll w, T cost) { // edge from u to v
with capacity w and cost
        edge a = edge(v, sz(g[v]) + (u == v), 0, w, cost, false);
 // + (u == v): self-loop
        edge b = edge(u, sz(g[u]), 0, 0, -cost, true);

        g[u].push_back(a);
        g[v].push_back(b);
    }

   vector<T> spfa(ll s) { // not needed if no negative cost edges
        deque<ll> q;
        vector<bool> is_inside(sz(g), 0);
        dist = vector<T>(sz(g), inf);

        dist[s] = 0;
        q.push_back(s);
        is_inside[s] = true;

        while (!q.empty()) {
            ll v = q.front();
            q.pop_front();
            is_inside[v] = false;

            forn(i,0,sz(g[v])){
                auto [to, rev, flow, cap, res, cost] = g[v][i];
                if (flow < cap && dist[v] + cost < dist[to]) {
                    dist[to] = dist[v] + cost;

                    if (is_inside[to]) continue;
                    if (!q.empty() && dist[to] > dist[q.front()])
 q.push_back(to);
                    else q.push_front(to);
                    is_inside[to] = true;
                }
            }
        }
        return dist;
    }
    bool dijkstra(ll s, ll t, vector<T>& pot) {
        priority_queue<pair<T, ll>, vector<pair<T, ll>>, greater
<>> q;
        dist = vector<T>(sz(g), inf);
        dist[s] = 0;
        q.emplace(0, s);
        while (sz(q)) {
            auto [d, v] = q.top();
            q.pop();
            if (dist[v] < d) continue;
            forn(i,0,sz(g[v])) {
                auto [to, rev, flow, cap, res, cost] = g[v][i];
                cost += pot[v] - pot[to];
                if (flow < cap && dist[v] + cost < dist[to]) {
                    dist[to] = dist[v] + cost;
                    q.emplace(dist[to], to);
                    par_idx[to] = i, par[to] = v;
                }
            }
        }
        return dist[t] < inf;
    }

    pair<ll, T> min_cost_flow(ll s, ll t, ll flow = INF) {
        vector<T> pot(sz(g), 0);
        pot = spfa(s); // change shortest path algorithm here (
pot = 0 se nao usar spfa)
        // no negative costs: nothing needed
        // DAG: use DP instead
        ll f = 0;
        T ret = 0;
        while (f < flow && dijkstra(s, t, pot)) {
            forn(i,0,sz(g))
                if (dist[i] < inf) pot[i] += dist[i];

            ll mn_flow = flow - f, u = t;
            while (u != s){
                mn_flow = min(mn_flow,
                    g[par[u]][par_idx[u]].cap - g[par[u]][
par_idx[u]].flow);
                u = par[u];
            }

            ret += pot[t] * mn_flow;

            u = t;
            while (u != s) {
                g[par[u]][par_idx[u]].flow += mn_flow;
                g[u][g[par[u]][par_idx[u]].rev].flow -= mn_flow;
                u = par[u];
            }

            f += mn_flow;
        }

        return make_pair(f, ret);
    }

    // Optional: returns original edges where flow == cap (
saturadas)
    // para arestas com fluxo parcial, troque por e.flow > 0
    vector<pll> recover() {
        vector<pll> used;
        forn(i, 0, sz(g)) for (edge e : g[i])

            if(e.flow == e.cap && !e.res) used.push_back({i, e.to});
        return used;
    }
```

```
c04 };
```

4.14 Strongly Connected Components (Kosaraju) c8c

Computes SCCs using two DFS passes and builds the condensation graph. roots[v] = id of v's SCC; ids are in topological order of the condensation (every edge goes from a smaller id to a larger one). Vertices 0-indexed. Recursive DFS: a path of ~1.3e5 vertices overflows an 8MB stack.

Complexity: O(N + M log M), O(N + M)

```
591 vector<bool> visited;

775 void dfs(ll v, vector<vll>& g, vll &out) {
e75     visited[v] = true;
819     for(auto u : g[v]) if(!visited[u]) dfs(u, g, out);
3ad     out.push_back(v);
043 }

cf2 vector<vll> scc(vector<vll>& g, vll& roots) {
625     int n = sz(g);
222     vll order; roots.assign(n, 0);

4cc     vector<vll> adj_rev(n);
0c2     forn(u, 0, n) for (ll v : g[u]) adj_rev[v].push_back(u);

411     visited.assign(n, false);
2b4     forn(i, 0, n) if (!visited[i]) dfs(i, g, order);
b3a     reverse(order.begin(), order.end());

411     visited.assign(n, false);
de0     ll curr_comp = 0;
0ee     for (auto v : order) {
451         if (!visited[v]) {
291             vll component; dfs(v, adj_rev, component);
fe3             for (auto u : component) roots[u] = curr_comp;
5f2             curr_comp++;
e3f         }
d55     }

e94     set<pll> edges;
361     vector<vll> cond_g(curr_comp);
c6b     forn(u, 0, n) {
7b9         for (auto v : g[u]) {
2dd             if (roots[u] != roots[v] && !edges.count({roots[u],
roots[v]})) {
2b9                 cond_g[roots[u]].push_back(roots[v]);
893                 edges.emplace(roots[u], roots[v]);
fbe             }
f76         }
3b9     }
594     return cond_g;
a22 }
```

4.15 Topological Sort (Kahn's Algorithm) ced

Produces a topological ordering of a DAG using indegree counting and a queue-like frontier. If g has a cycle, returns fewer than sz(g) vertices (cycle iff sz(result) < n).

Complexity: O(N + M), O(N + M)

```
af3 vll topo_sort(const vector<vll>& g) {
a93     vll indeg(sz(g)), q;
46b     for (auto& li : g) for (ll x : li) indeg[x]++;
707     forn(i, 0, sz(g)) if (indeg[i] == 0) q.push_back(i);
c4b     forn(j, 0, sz(q)) for(ll x : g[q[j]]) if(--indeg[x] == 0) q.
push_back(x);
bef     return q;
ced }
```

4.16 Virtual Tree 8b0

Comprime a arvore num conjunto S: o resultado contem S, e fechado por LCA e tem menos de 2|S| vertices. S nao pode ser vazio. Retorna a raiz. virt[u] = arestas descendo {filho, distancia em arestas}; virt deve ter tamanho n e so as listas dos vertices usados sao limpas. Precisa de lca.cpp (lca::time, lca::lca, lca::dist).

Complexity: O(k log k + k * T), T = tempo do lca::lca

```
aa4 #include "src/graphs/lca.cpp"

6ce ll build_virt(vll v, LCA& lca, vector<vector<pll>>& virt) {
06b     auto cmp = [&](ll i, ll j) { return lca.time[i] < lca.time[j
]; };
074     sort(v.begin(), v.end(), cmp);
f4d
    for (ll i = sz(v)-1; i; i--) v.push_back(lca.lca(v[i], v[i-1]));
074     sort(v.begin(), v.end(), cmp);
d76     v.erase(unique(v.begin(), v.end()), v.end());
9e2     forn(i,0,sz(v)) virt[v[i]].clear();
c03     forn(i,1,sz(v)) {
d33         ll parent = lca.lca(v[i-1], v[i]);
b82         ll d = lca.dist(parent, v[i]);
 // #warning soh to colocando aresta descendo
4d0         virt[parent].emplace_back(v[i], d);
1e4     }

832     return v[0];
141 }
```

5 Mathematics

5.1 Chinese Remainder Theorem d6d

Dados x = a (mod m) e x = b (mod n), com 0 <= a < m e 0 <= b < n, acha o unico c < mmc(m, n) com x = c (mod mmc(m, n)). Existe se e so se mdc(m, n) divide b - a; senao retorna -INF.

Complexity: O(log(min(m, n)))

```
b12 #include "src/math/euclid.cpp"

24a ll crt(ll a, ll m, ll b, ll n) {
08f     if (n > m) swap(a, b), swap(m, n);

58f     ll x, y, g = euclid(m, n, x, y);
51a     if ((a - b) % g != 0) return -INF; // não tem solução

110     __int128 k = (__int128)((b-a)/g) * x;
7a9     k %= n/g;
c7d     if (k < 0) k += n/g;

cfd     return (ll)((__int128)m*k + a);
f20 }
```

5.2 Extended Euclidean Algorithm 33b

Retorna g = mdc(a,b) e encontra x,y tais que ax+by=g. Requer a,b >= 0.

```
c22 ll euclid(ll a, ll b, ll &x, ll &y) {
1ee     if (!b) return x = 1, y = 0, a;
e3d     ll d = euclid(b, a % b, y, x);
0a4     return y -= a/b * x, d;
33b }
```

5.3 NTT Convolution a0c

Implements iterative NTT over supported primes. Supports convolution of two vectors.

Complexity: O(N log N) time, O(N) memory.

```
360 #include "src/math/mint.cpp" // for nnt

6c0 template<ll p>
206 void get_roots(bool f, ll n, vector<mod_int<p>>& roots) {
1e6     mod_int<p> r;
bfe     ll ord;

57a     if (p == 998244353) {
9b6         r = 102292;
12c         ord = (1LL << 23);
e32     } else if (p == 754974721) {
43a         r = 739831874;
207         ord = (1LL << 24);
877     } else if (p == 167772161) {
a2a         r = 243;
8ba         ord = (1LL << 25);
7a8     } else {
797         assert(false);
a73     }

1e5     assert(n <= ord && ord % n == 0);

547     if (f) r = r^(p - 1 - ord/n);
ee2     else r = r^(ord/n);

be4     roots[0] = 1;
098     forn(i,1,n/2) roots[i] = roots[i-1]*r;
216 }

67a template<typename T>
e60 void fft(vector<T>& a, bool f, ll N, vector<ll>& rev) {
ffd     assert(N > 0 && (N & (N-1)) == 0);
e08     assert(sz(a) == N && sz(rev) == N);

73b     forn(i,0,N) if (i < rev[i]) swap(a[i], a[rev[i]]);

816     ll l, r, m;
cb4     vector<T> roots(N);

0ec     for (ll n = 2; n <= N; n *= 2) {
0f4         get_roots(f, n, roots);

dc1         for (ll pos = 0; pos < N; pos += n) {
aee             l = pos, r = pos + n/2, m = 0;

a88             while (m < n/2) {
297                 auto t = roots[m]*a[r];
254                 a[r] = a[l]-t;
b8f                 a[l] = a[l]+t;
2c9                 l++, r++, m++;
d89             }
6ed         }
738     }

235     if (f) {
1c5         auto invN = T(1)/T(N);
2a8         forn(i,0,N) a[i] *= invN;
c0b     }
```

```
4d1 }

67a template<typename T>
381 vector<T> convolution(vector<T> l, vector<T> r) {
701     if (l.empty() || r.empty()) return {};

3f1     ll N = sz(l)+sz(r)-1;
a10     ll n = 1, log_n = 0;

8cf     while (n < N) n *= 2, log_n++;

e66     vector<ll> rev(n);

522     forn(i,0,n) {
434         rev[i] = 0;
0d7         forn(j,0,log_n) if (i>>j&1)
bad             rev[i] |= 1LL << (log_n-1-j);
0a4     }

fa4     l.resize(n);
7e4     r.resize(n);

56e     fft(l, false, n, rev);
fcf     fft(r, false, n, rev);

a9b     forn(i,0,n) l[i] *= r[i];

88b     fft(l, true, n, rev);

5e1     l.resize(N);
792     return l;
c0a }

// NTT: converts the input vectors to mod_int<p>.
4a7 template<ll p, typename T>
e0a vector<mod_int<p>> ntt(const vector<T>& a, const vector<T>& b) {
2b2     vector<mod_int<p>> A(a.begin(), a.end());
726     vector<mod_int<p>> B(b.begin(), b.end());

ffd     return convolution(move(A), move(B));
1c3 }
```

5.4 Gauss - Z2

fd2

*D eh a dimensao maxima do espaco vetorial. add(v): insere v na base. Retorna true se v eh independente dos vetores anteriores. coord(v): retorna as coordenadas de v na base atual. basis^T * c = v, considerando soma XOR. recover(v): retorna os indices dos vetores originalmente inseridos cuja soma XOR eh v. Os indices sao 0-based e ordenados. coord(v).first e recover(v).first indicam se v pertence ao span. add, coord, recover: O(D^2 / 64) Memory: O(D^2 / 64)*

```
9d5 template<ll D>
a2e struct gauss_z2 {
3c1     bitset<D> basis[D], keep[D];
1cf     ll rk, in, dim;
d6f     vll id;

82a     gauss_z2() : rk(0), in(-1), dim(D), id(D, -1) {}

6b8     gauss_z2(ll dim_) : rk(0), in(-1), dim(dim_), id(D, -1) {
859         assert(0 <= dim && dim <= D);
0d0     }

2bd     void check_dim(const bitset<D>& v) const {
a5a         forn(i,dim,D) assert(!v[i]);
0dc     }

04e     bool add(bitset<D> v) {
b55         check_dim(v);
42c         in++;

fb0         bitset<D> k;

294         for (ll i = dim-1; i >= 0; i--) if (v[i]) {
654             if (basis[i][i]) {
647                 v ^= basis[i];
17e                 k ^= keep[i];
9e3             } else {
f5f                 k[i] = true;
cbf                 id[i] = in;
901                 keep[i] = k;
d67                 basis[i] = v;
92a                 rk++;
8a6                 return true;
dd0             }
bb2         }

d1f         return false;
083     }

7af     pair<bool, bitset<D>> coord(bitset<D> v) const {
b55         check_dim(v);

944         bitset<D> c;

294         for (ll i = dim-1; i >= 0; i--) if (v[i]) {
654             if (basis[i][i]) {
647                 v ^= basis[i];
69e                 c[i] = true;
d60             } else {
d42                 return {false, bitset<D>()};
cf4             }
501         }

5db         return {true, c};
9b8     }

f9e     pair<bool, vll> recover(bitset<D> v) const {
22e         auto [span, bc] = coord(v);

d76         if (!span) return {false, {}};

f79         bitset<D> aux;

722         for (ll i = dim-1; i >= 0; i--)
f55             if (bc[i]) aux ^= keep[i];

79a         vll oc;

722         for (ll i = dim-1; i >= 0; i--)
110             if (aux[i]) oc.push_back(id[i]);

e08         sort(oc.begin(), oc.end());

001         return {true, oc};
546     }
84d };
```

5.5 Gauss-Jordan

bd0

Resolve um sistema linear Ax = b. Retorna: {0, {}} se nao existe solucao; {1, ans} se existe solucao unica; {INF, ans} se existem infinitas solucoes. Para tipos de ponto flutuante (double, long double). eps e uma tolerancia absoluta.
Complexity: O(n^2 * m) time, O(n*m) memory.

```
67a template<typename T>
110 pair<ll, vector<T>> gauss(
2cb     vector<vector<T>> a,
c1f     vector<T> b,
4bc     long double eps = 1e-9L
0a2 ) {
50a     ll n = sz(a);
e22     assert(n > 0 && sz(b) == n);

c8b     ll m = sz(a[0]);
be2     for (auto &row : a) assert(sz(row) == m);

f4c     forn(i,0,n) a[i].push_back(b[i]);

e47     vll where(m, -1);

d41     for (ll col = 0, row = 0; col < m && row < n; col++) {
b49         ll sel = row;

d96         forn(i,row,n)
24a             if (abs(a[i][col]) > abs(a[sel][col]))
403                 sel = i;

db0         if (abs(a[sel][col]) <= eps) continue;

6b4         swap(a[sel], a[row]);
2c3         where[col] = row;

959         forn(i,0,n) if (i != row) {
96c             T c = a[i][col] / a[row][col];

b2d             forn(j,col,m+1)
c8f                 a[i][j] -= a[row][j]*c;
5a4         }

b70         row++;
715     }

b1d     vector<T> ans(m, 0);

363     forn(i,0,m) if (where[i] != -1)
12a         ans[i] = a[where[i]][m]/a[where[i]][i];

522     forn(i,0,n) {
501         T sum = 0;

695         forn(j,0,m)
5a9             sum += ans[j]*a[i][j];

b1f         if (abs(sum-a[i][m]) > eps)
7d6             return {0, {}};
9d0     }

688     forn(i,0,m) if (where[i] == -1)
e2e         return {INF, ans};

d4a     return {1, ans};
```

```
013 }
```

5.6 Matrix template caa

operator: multiplicacao; modo modular com #define MODULAR true. operator^: exponenciacao rapida. apply_transform(): calcula M^e * v.*

Complexity: operator* O(n m r), operator^ O(n^3 log e)

```
945 #define MODULAR false
5ed template<typename T> struct matrix : vector<vector<T>> {
03e     ll n, m;
30f     void print() {
522         forn(i,0,n) {
064             forn(j,0,m) cout << (*this)[i][j] << " ";
1fb             cout << endl;
285         }
823     }

491     matrix(ll n_, ll m_, bool ident = false) :
b14
            vector<vector<T>>(n_, vector<T>(m_, 0)), n(n_), m(m_) {
94e         if (ident) {
df7             assert(n == m);
17a             forn(i,0,n) (*this)[i][i] = 1;
636         }
f73     }
b83     matrix(const vector<vector<T>>& c) : vector<vector<T>>(c),
98e         n(sz(c)), m(sz(c[0])) {}
efc     matrix(const initializer_list<initializer_list<T>>& c) {
f7e         vector<vector<T>> val;
212         for (auto& i : c) val.push_back(i);
303         *this = matrix(val);
c50     }

388     matrix<T> operator*(matrix<T>& r) {
1e2         assert(m == r.n);
82c         matrix<T> M(n, r.m);
630         forn(i,0,n) forn(k,0,m) forn(j, 0, r.m){
e34             T add = (*this)[i][k] * r[k][j];
f98 #if MODULAR
d41 #warning Usar matrix<ll> e soh colocar valores em [0, MOD) na
matriz!
8b6             M[i][j] += add%MOD;
983             if (M[i][j] >= MOD) M[i][j] -= MOD;
8c1 #else
7bb             M[i][j] += add;
f2e #endif
77d         }
474         return M;
ba1     }
528     matrix<T> operator^(ll e){
f10         matrix<T> M(n, n, true), at = *this;
c87         while (e) {
2e2             if (e&1) M = M*at;
cc2             e >>= 1;
c80             at = at*at;
eb6         }
474         return M;
ca3     }
582     void apply_transform(matrix M, ll e){
1c3         auto& v = *this;
c87         while (e) {
9ba             if (e&1) v = M*v;
cc2             e >>= 1;
419             M = M*M;
d86         }
4e5     }
f02 };
```

5.7 Miller-Rabin Primality Test (Deterministic for 64-bit) 60d

Checks whether n is prime using the seven deterministic bases: {2, 325, 9375, 28178, 450775, 9780504, 1795265022}. Correct for every n < 2^64. Requires: - modmul and modpow from mod_functions.cpp. - Exact modular multiplication. - O(log n) modular multiplications per query (7 bases).

```
da4 bool isPrime(ull n){
c16     if(n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
ef3     ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
 s = __builtin_ctzll(n-1), d = n >> s;
e80     for (ull a : A) {
6b4         ull p = modpow(a%n, d, n), i = s;
274         while(p != 1 && p != n-1 && a % n && i--)
c77             p = modmul(p, p, n);
e28         if(p != n-1 && i != s) return 0;
edf     }
6a5     return 1;
60d }
```

5.8 Modular Integer 5b2

*Fixed-modulus integer type with +, -, *, /, and exponentiation. Division and negative exponents use Fermat's little theorem; therefore, the modulus must be prime. Preconditions: - p > 1. - p is prime for division and negative exponents. - p^2 fits in ll. - Exponents must not be LLONG_MIN. - +, -, *: O(1). - /: O(log p). - ^: O(log |e|).*

```
f25 template<ll p> struct mod_int {
97f     static_assert(p > 1);
0be     static_assert((__int128)p * p <= LLONG_MAX);

62c     ll expo(ll b, ll e) const {
c85         ll ret = 1;
c87         while (e) {
cad             if (e % 2) ret = ret * b % p;
9d2             e /= 2, b = b * b % p;
c42         }
edf         return ret;
df3     }

a67     ll inv(ll b) const {
bf9         assert(b != 0);
61d         return expo(b, p-2);
2f7     }

4d7     using m = mod_int;
aa3     ll v;

fe0     mod_int() : v(0) {}

e12     mod_int(ll v_) {
c8e         if (v_ >= p || v_ <= -p) v_ %= p;
bc6         if (v_ < 0) v_ += p;
2e7         v = v_;
5d7     }

74d     m& operator +=(const m& a) {
2fd         v += a.v;
ba5         if (v >= p) v -= p;
357         return *this;
c8b     }

eff     m& operator -=(const m& a) {
8b4         v -= a.v;
cc8         if (v < 0) v += p;
357         return *this;
f8d     }

4c4     m& operator *=(const m& a) {
686         v = v * a.v % p;
357         return *this;
c9b     }

3f9     m& operator /=(const m& a) {
b43         assert(a.v != 0);
5d6         v = v * inv(a.v) % p;
357         return *this;
d5e     }

f0a     m operator -() const {
0c1         return m(-v);
f98     }

b3e     m& operator ^=(ll e) {
edc         assert(e != LLONG_MIN);
06d         if (e < 0) {
6e2             v = inv(v);
00c             e = -e;
275         }
284         v = expo(v, e);
357         return *this;
b4c     }

149     bool operator ==(const m& a) const {
7ad         return v == a.v;
dd1     }

de5     bool operator !=(const m& a) const {
172         return v != a.v;
220     }

1c6     friend istream& operator >>(istream& in, m& a) {
348         ll val;
5e9         in >> val;
d48         a = m(val);
091         return in;
870     }

44f     friend ostream& operator <<(ostream& out, m a) {
5a0         return out << a.v;
214     }

399     friend m operator +(m a, m b) { return a += b; }
f9e     friend m operator -(m a, m b) { return a -= b; }
9c1     friend m operator *(m a, m b) { return a *= b; }
51b     friend m operator /(m a, m b) { return a /= b; }
08f     friend m operator ^(m a, ll e) { return a ^= e; }
5b2 };
```

5.9 Modular Arithmetic Helpers 8b4

*modmul(a, b, M): computes (a*b) % M exactly. modpow(b, e, M): computes b^e % M using binary exponentiation. Preconditions: - M > 0. - modmul supports arbitrary*

ull operands. - modpow returns a value in [0, M). Uses __int128 for exact 64-bit modular multiplication. - modmul: O(1) machine operations. - modpow: O(log e) modular multiplications.

```
f4c typedef unsigned long long ull;

f85 ull modmul(ull a, ull b, ull M) {
f9f     return (unsigned __int128)a * b % M;
ccb }

4f6 ull modpow(ull b, ull e, ull mod){
c1a     ull ans = 1;
a18     for(; e; b = modmul(b, b, mod), e /= 2)
9e8         if(e & 1) ans = modmul(ans, b, mod);
ba7     return ans;
100 }
```

5.10 Discrete Logarithm (Baby-step Giant-step) d59

*Finds the smallest x >= 1 such that a^x = b (mod m), or returns -1 if no solution exists. Works with composite moduli and does not require gcd(a,m) = 1. Preconditions: - m > 0. Uses __int128 for exact modular multiplication. - Time: O(sqrt(m) * log m). - Memory: O(sqrt(m)).*

```
a4f ll mod_log(ll a, ll b, ll m) {
0a6     assert(m > 0);

ec0     a %= m;
d35     b %= m;
700     if (a < 0) a += m;
576     if (b < 0) b += m;

9ae     ll n = (ll)sqrtl((long double)m) + 1;

7da     auto mul = [&](ll x, ll y) -> ll {
a2a         return (ll)((__int128)x * y % m);
a40     };

82c     ll e = 1 % m, f = 1 % m, j = 1;
e2d     map<ll, ll> mp;

4bc     while (j <= n) {
ab1         e = f = mul(e, a);
8e4         if (e == b) return j;
625         mp[mul(e, b)] = j++;
9ee     }

3c3     if (gcd(m, e) == gcd(m, b)) {
04f         forn(i, 2, n+2) {
a12             e = mul(e, f);
bf1             if (mp.count(e))
b6c                 return (ll)((__int128)n * i - mp[e]);
7d9         }
07b     }

daa     return -1;
d59 }
```

5.11 Discrete Square Root 197

Tonelli-Shanks: finds x such that x^2 = a (mod p), or returns -1 if no solution exists. Preconditions: - p is prime. - 2 <= p <= LLONG_MAX. Uses exact modular multiplication through mod_functions.cpp. - Tonelli-Shanks: O(log^2 p) modular operations. - Finding a quadratic non-residue: additional O(q log p), where q is the number of candidates tested.

```
924 #include "src/math/mod_functions.cpp"

29c ll mod_sqrt(ll a, ll p) {
5fe     assert(p >= 2);

58c     a %= p;
752     if (a < 0) a += p;

c9c     if (p == 2) return a;
b47     if (a == 0) return 0;

709     if (modpow(a, (p-1)/2, p) != 1) return -1;

bd6     if (p % 4 == 3)
b2a         return modpow(a, p/4 + 1, p);

7da     auto mul = [&](ll x, ll y) -> ll {
e0a         return (ll)((__int128)x * y % p);
4a0     };

f69     ll s = p-1, z = 2;
ee5     int r = 0, m;

084     while (s % 2 == 0)
082         ++r, s /= 2;

    // Find a quadratic non-residue.
453     while (modpow(z, (p-1)/2, p) != p-1)
4f0         ++z;

0c3     ll x = modpow(a, (s+1)/2, p);
6a6     ll b = modpow(a, s, p);
f3a     ll g = modpow(z, s, p);

1de     for (;;) {
4fd         ll t = b;

713         for (m = 0; m < r && t != 1; ++m)
075             t = mul(t, t);

ae0         if (m == 0) return x;

5a8         assert(m < r);

20e         ll gs = modpow(g, 1LL << (r-m-1), p);

56b         g = mul(gs, gs);
813         x = mul(x, gs);
8e3         b = mul(b, g);
c21     }
394 }
```

5.12 Primitive Root (Generator) 4ce

Finds the smallest primitive root modulo a prime P. Factorizes P-1 and checks that for every prime factor F of P-1, the candidate G satisfies G^((P-1)/F) != 1 (mod P). can be adapted to p not prime

Complexity: O(time_to_factor(p-1) + ans * |factors(P-1)| * log P)

```
924 #include "src/math/mod_functions.cpp"

e7f ll primitive_root(ll p){
1e7     vll fact;
546     ll phi = p-1,  n = phi;
    // mudar p-1 para caso nao primo
b6d     for (ll i=2; i*i<=n; i++)
c06         if (n % i == 0) {
a54             fact.push_back (i);
49e             while (n % i == 0)
135                 n /= i;
7a0         }
f3d     if (n > 1)
d24         fact.push_back(n);

293     forn(res,2, p+1) {
971         bool ok = true;
557         for (ll i=0; i<sz(fact) && ok; ++i)
754             ok &= modpow (res, phi / fact[i], p) != 1;
97c         if (ok)  return res;
504     }
daa     return -1;
23e }
```

5.13 Quadratic Field Extension (Z_p[sqrt(D)]) b77

*Extension for numbers of form x + y*sqrt(D) with +, -, *, /, and exponentiation.*

Complexity: O(1) per arithmetic op, O(log MOD) for division/inversion.

```
360 #include "src/math/mint.cpp"

41a template <long long D>
281 struct quad {
14f     mint x, y;
f0c     using m = quad<D>;

c72     quad() : x(0), y(0) {}
565     quad(ll x_) : x(x_), y(0) {}
9e9     quad(mint x_, mint y_) : x(x_), y(y_) {}

74d     m& operator += (const m& a) {
12b         x += a.x; y += a.y; return *this;
30f     }
eff     m& operator -= (const m& a) {
dd8         x -= a.x; y -= a.y; return *this;
a07     }

4c4     m& operator *= (const m& a) {
86a         mint old_x = x;
1df         x = x * a.x + mint(D) * y * a.y;
275         y = old_x * a.y + y * a.x;
357         return *this;
099     }

ad4     m inv() const {
04b         mint dem = x * x - mint(D) * y * y;
117         return m(x / dem, -y / dem);
2b5     }

3f9     m& operator /=(const m& a) {
8f8         *this *= a.inv();
357         return *this;
201     }

8de     m expo(m b, ll e){
7f9         m ret = m(1, 0);
c87         while (e) {
9a9             if (e % 2) ret = ret * b;
e04             b = b * b;
```

```
e1c             e /= 2;
9a5         }
edf         return ret;
bc1     }

b3e     m& operator ^=(ll e) {
06d         if (e < 0) {
625             *this = this->inv();
00c             e = -e;
019         }
670         *this = expo(*this, e);
357         return *this;
7b9     }

1c2
    bool operator ==(const m& a){ return (x == a.x) && (y == a.y); }
86b
    bool operator !=(const m& a){ return (x != a.x) || (y != a.y); }

b12     friend ostream& operator <<(ostream& out, const m& a) {
7fb         return out << a.x << ":" << a.y;
1b1     }

5a6     friend m operator +(m a, const m& b) { return a += b; }
022     friend m operator -(m a, const m& b) { return a -= b; }
cfb     friend m operator *(m a, const m& b) { return a *= b; }
a1e     friend m operator /(m a, const m& b) { return a /= b; }
23d     friend m operator ^(m a, ll b) { return a ^= b; }
298 };
```

5.14 Linear Sieve (prime precomputation) 70c

Computes primes up to N in O(N) using the smallest prime factor (lp). pr will contain all primes <= N. lp[x] stores the smallest prime dividing x.

Complexity: O(N)

```
936 pair<vll, vll> sieve(ll n){
309     vll lp(n+1);
e67     vll pr;
3e5     forn(i, 2, n+1) {
719         if (lp[i] == 0) {
116             lp[i] = i;
bc0             pr.push_back(i);
d4f         }
778         for (ll j = 0; i * pr[j] <= n; ++j) {
a97             lp[i * pr[j]] = pr[j];
5f9             if (pr[j] == lp[i]) {
c2b                 break;
e08             }
9b9         }
8d6     }
52d     return {pr, lp};
728 }

638 vll calc_mu(vll& lp) {
c96     ll n = sz(lp) - 1;
caf     vll ans(n);
ca9     ans[1] = 1;
aff     forn(i,2,n) {
5ea         ll p = lp[i], x = i/p;
08c         if (lp[x] == p) ans[i] = 0;
bcf         else ans[i] = -ans[x];
29f     }
ba7     return ans;
bc2 }

823 vll calc_d(vll& lp) {
c96     ll n = sz(lp) - 1;
caf     vll ans(n);
ca9     ans[1] = 1;
aff     forn(i, 2, n) {
5ea         ll p = lp[i], x = i / p;
f85         if (lp[x] != p) ans[i] = 1;
4f9         else ans[i] = ans[x] + 1;
cc9     }
aff     forn(i, 2, n) {
5ea         ll p = lp[i], x = i / p;
055         if (ans[i] == 1) ans[i] = ans[x] * 2;
b3f         else ans[i] = (ans[x] / ans[i]) * (ans[i] + 1);
bff     }
ba7     return ans;
803 }
```

6 String Algorithms

6.1 Aho-Corasick Automaton 7d0

Trie with failure links for multi-pattern matching. Node v = string from the root to v. After build(): link[v] = longest proper suffix of v in the trie (-1 for root); ext[v] = nearest pattern end in the link chain (-1 if none); term[v] = patterns ending exactly at v; cnt[v] = patterns that are suffixes of v; mx[v] = length of the longest pattern that is a suffix of v (0 if none). Patterns matched at v: follow v, ext[v], ext[ext[v]], ... (skip v if !term[v]).

Complexity: O((sum |patterns| + |text|) log sigma)

```
60c struct aho {
552     struct node { map<char, ll> to; ll link = 0, ext = -1, term
= 0, cnt = 0, mx = 0; };
7d8     vector<node> t = {node{{}, -1}};

e7d     void insert(const string& s) {
4eb         ll at = 0;
b4f         for (char c : s) {
034             if (!t[at].to.count(c)) t[at].to[c] = sz(t), t.
emplace_back();
e20             at = t[at].to[c];
ef3         }
3d7         t[at].term++, t[at].cnt++, t[at].mx = sz(s);
53e     }

6dd     ll next(ll at, char c) {
ffe         while (at != -1 && !t[at].to.count(c)) at = t[at].link;
d81         return at == -1 ? 0 : t[at].to[c];
54b     }

0a8     void build() {
68d         queue<ll> q; q.push(0);
647         while (sz(q)) {
aa7             ll i = q.front(); q.pop();
1b6             for (auto [c, j] : t[i].to) {
66d                 ll l = t[j].link = next(t[i].link, c);
ef2                 t[j].ext = t[l].term ? l : t[l].ext;
380                 t[j].cnt += t[l].cnt, t[j].mx = max(t[j].mx, t[l
].mx);
113                 q.push(j);
921             }
8ed         }
19a     }

    // total number of pattern occurrences in s
393     ll query(const string& s) {
0db         ll at = 0, ans = 0;
915         for (char c : s) at = next(at, c), ans += t[at].cnt;
ba7         return ans;
963     }

    // v[i] = length of the longest pattern ending at position i (0
 if none)
4e6     vll match_vec(const string& s) {
c83         ll at = 0, n = sz(s);
d26         vll v(n, 0);
59d         forn(i, 0, n) at = next(at, s[i]), v[i] = t[at].mx;
6dc         return v;
314     }
7d0 };
```

6.2 Knuth-Morris-Pratt (KMP) 459

Computes prefix function and performs linear-time substring search with optional automaton construction.

Complexity: O(n + m), O(n)

```
b9d vll pi(string& s) {
36f     vll p(sz(s));
3b9     for (ll i = 1, j = 0; i < sz(s); i++) {
5c2         while (j && s[j] != s[i]) j = p[j-1];
973         if (s[j] == s[i]) j++;
f8c         p[i] = j;
00d     }
74e     return p;
bac }

ce5 vll match(string& pat, string& s) {
2b9     vll p = pi(pat), match;
c0f     for (ll i = 0, j = 0; i < sz(s); i++) {
6e6         while (j && pat[j] != s[i]) j = p[j-1];
3e3         if (pat[j] == s[i]) j++;
1f4         if (j == sz(pat)) match.push_back(i-j+1), j = p[j-1];
ae5     }
ed8     return match;
07e }

2c7 struct KMPaut : vector<vll> {
47c     KMPaut(){}
bb4     KMPaut (string& s) : vector<vll>(26, vll(sz(s)+1)) {
411         vll p = pi(s);
04b         auto& aut = *this;
4fa         aut[s[0]-'a'][0] = 1;
84b         forn(c, 0, 26) forn(i, 1, sz(s)+1)
42b             aut[c][i] = s[i]-'a' == c ? i+1 : aut[c][p[i-1]];
940     }
42e };
```

6.3 Suffix Array - O(n log n) 039

kasai recebe o suffix array e calcula lcp[i] = lcp entre os sufixos s[sa[i]..] e s[sa[i+1]..].

Complexity: suffix_array O(n log n), kasai O(n)

```
f32 vll suffix_array(string& s) {
0a2     ll n = sz(s)+1, N = max(n, 257ll);
eaf     vll sa(n), ra(n);
```

```
828     forn(i, 0, n) sa[i] = i, ra[i] = s[i];

2e6     for(ll k = 0; k < n; k ? k *= 2 : k++) {
dad         vll nsa(sa), nra(n), cnt(N);

d83         forn(i, 0, n) nsa[i] = (nsa[i]-k+n)%n, cnt[ra[i]]++;
e18         forn(i, 1, N) cnt[i] += cnt[i-1];
b97
            for(ll i = n-1; i >= 0; i--) sa[--cnt[ra[nsa[i]]]] = nsa[i];

aec         forn(ll i = 1, r = 0; i < n; i++) nra[sa[i]] = r += ra[sa
[i]] !=
c45             ra[sa[i-1]] || ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)%n];
26b         ra = nra;
d5e         if (ra[sa[n-1]] == n-1) break;
b60     }
649     return vll(sa.begin()+1, sa.end());
c06 }

ff6 vll kasai(string& s, vll& sa) {
c30     ll n = sz(s), k = 0;
1b5     vll ra(n), lcp(n);
540     forn(i, 0, n) ra[sa[i]] = i;

514     for (ll i = 0; i < n; i++, k -= !!k) {
199         if (ra[i] == n-1) { k = 0; continue; }
674         ll j = sa[ra[i]+1];
329         while (i+k < n && j+k < n && s[i+k] == s[j+k]) k++;
d98         lcp[ra[i]] = k;
b76     }
5ed     return lcp;
8c9 }
```

6.4 Trie (Prefix Tree) ffd

*Stores strings over a fixed alphabet to support insert, erase, and prefix counting in linear time. to[x*sigma + c] = child of node x by char c (0 = none). Root is node 0. erase(s) does nothing and returns false if s is not in the trie.*

Complexity: O(|s|) per op, O(sigma * total inserted chars) memory

```
ab5 struct trie {
046     vll to, end, pref;
1c5     ll sigma; char norm;

d74     trie(ll sigma_=26, char norm_='a') : to(sigma_), end(1),
pref(1), sigma(sigma_), norm(norm_) {}

bfc     void insert(string &s) {
00d         ll x = 0;
7e7         for (auto c : s) {
1af             ll id = x*sigma + c-norm;
881             if (!to[id]) {
813                 to[id] = sz(end);
ef0                 to.resize(to.size() + sigma);
770                 end.push_back(0), pref.push_back(0);
2a2             }
fe0             x = to[id], pref[x]++;
232         }
421         end[x]++, pref[0]++;
a70     }

5c9     bool erase(string &s) {
78e         ll x = find(s);
e74         if (x < 0 || !end[x]) return false;
ec2         x = 0;
7e7         for (auto c : s) {
dc5             ll &nxt = to[x*sigma + c-norm];
f2d             x = nxt;
c74             if (!--pref[x]) nxt = 0;
1d1         }
104         end[x]--, pref[0]--;
8a6         return true;
36f     }

b48     ll find(string &s) {
00d         ll x = 0;
7e7         for (auto c : s) {
17a             x = to[x*sigma + c-norm];
59b             if (!x) return -1;
3fa         }
ea5         return x;
b67     }

d96     ll count_pref(string &s) {
b09         ll id = find(s);
fc1         return id >= 0 ? pref[id] : 0;
4be     }
ffd };
```

6.5 Z-function 8ea

z[i] = lcp between s[i..n) and s.

Complexity: O(n)

```
008 vll get_z(string s) {
3b8     ll n = sz(s);
0f9     vll z(n, 0);

837     ll l = 0, r = 0;
590     forn(i, 1, n) {
0af         if (i <= r) z[i] = min(r - i + 1, z[i - l]);
4ed         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
65e         if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
bd8     }

070     return z;
8ea }
```

7 Data Structures

7.1 Fenwick Tree with Range Updates db8

Complexity: O(log n) por op, O(n) build

```
bf2 struct Node {
60e     ll v = 0;
2dd     Node() {}
     //   Node() -> identidade
c52     Node(ll x) : v(x) {}
011
    Node operator+(const Node& o) const { return Node(v ^ o.v); }
     //   a + b -> operacao do grupo (associativa, comutativa)
09a     Node operator-() const { return *this; }
     //   -a -> inverso
ab5
    Node operator-(const Node& o) const { return Node(v ^ o.v); }
     //   a - b -> a + (-b)
e2e     Node operator*(ll k) const { return Node((k & 1) ? v : 0); }
    //   a * k (k: ll) -> a aplicado k vezes  (acao escalar, k e
indice)
296 };

a47 template <class T = ll>
f4c struct BitRange {
4de     ll n;
e32     vector<array<T, 2>> bit;  // {mul, add}
c3c     BitRange(ll _n = 0) : n(_n), bit(n + 2) {}
1ee     BitRange(vector<T>& v) : BitRange(sz(v)) {
f00         forn(i,1,n+1) {
076             bit[i][1] = bit[i][1] + v[i - 1];
c8f             ll j = i + (i & -i);
a21             if (j <= n) bit[j][1] = bit[j][1] + bit[i][1];
57b         }
1fe     }
9ea     void add(ll i, const T& m, const T& a) {
a6d         for (; i <= n; i += i & -i)
e9b
                bit[i][0] = bit[i][0] + m, bit[i][1] = bit[i][1] + a;
1ed     }
165     T pref(ll i) {  // combina [0, i-1]
36d         T m{}, a{};
da3         for (ll j = i; j; j -= j & -j)
cb5             m = m + bit[j][0], a = a + bit[j][1];
01f         return m * i + a;
e62     }
a94     void update(ll l, ll r, const T& x) {  // aplica x em [l, r]
335         add(l + 1, x, -(x * l));
688         add(r + 2, -x, x * (r + 1));
59c     }
ec3     T query(ll l, ll r) { return pref(r + 1) - pref(l); }
be5 };
```

7.2 Convex Hull Trick Dinamico 5ae

add(a, b): adiciona a reta ax + b. query(x): maximo de ax + b entre as retas. Para double, use INF = 1/.0 e div(a, b) = a/b.

Complexity: O(log n) amortizado por add, O(log n) por query

```
72c struct Line {
073     mutable ll a, b, p;
8e3     bool operator<(const Line& o) const { return a < o.a; }
abf     bool operator<(ll x) const { return p < x; }
469 };

1b7 struct CHT : multiset<Line, less<>> {
33a     ll div(ll a, ll b) {
353         return a / b - ((a ^ b) < 0 && a % b);
10f     }

bbb     void update(iterator x) {
459         if (next(x) == end()) x->p = INF;
eec         else if (x->a == next(x)->a) x->p = x->b >= next(x)->b ?
 INF : -INF;
424         else x->p = div(next(x)->b - x->b, x->a - next(x)->a);
d37     }

71c     bool overlap(iterator x) {
f18         update(x);
cfa         if (next(x) == end()) return 0;
a4a         if (x->a == next(x)->a) return x->b >= next(x)->b;
d40         return x->p >= next(x)->p;
901     }
```

```
    void add(ll a, ll b) {
        auto x = insert({a, b, 0});
        while (overlap(x)) erase(next(x)), update(x);
        if (x != begin() && !overlap(prev(x))) x = prev(x),
update(x);
        while (x != begin() && overlap(prev(x)))
            x = prev(x), erase(next(x)), update(x);
    }

    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
#warning cuidado com overflow!
        return l.a * x + l.b;
    }
};
```

7.3 DSU with Rollback

d99

Union-Find sem path compression para permitir desfazer uniões. t = time() salva um checkpoint; rollback(t) desfaz tudo depois dele. len[x] so e valido se x for raiz.

Complexity: O(log N) find/unite, O(1) amortizado por operacao desfeita

```
struct dsu_rb {
    vll id, len;
    stack<pair<ll&, ll>> st;

    dsu_rb(ll n) : id(n), len(n, 1) { iota(id.begin(), id.end(),
0); }

    void save(ll& x) { st.emplace(x, x); }

    ll time() { return sz(st); }

    void rollback(ll t) {
        while (sz(st) > t) {
            auto [ref, val] = st.top(); st.pop();
            ref = val;
        }
    }

    ll find(ll a) { return a == id[a] ? a : find(id[a]); }

    bool unite(ll a, ll b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (len[a] < len[b]) swap(a, b);
        save(len[a]), save(id[b]);
        len[a] += len[b], id[b] = a;
        return true;
    }
};
```

7.4 Disjoint Set Union (Union-Find)

b72

Supports find with path compression and union by size to maintain dynamic connectivity of disjoint sets.

Complexity: O(alpha(N)) amortized per op, O(N)

```
struct dsu {
    vll id, len;

    dsu(ll n) : id(n), len(n, 1) { iota(id.begin(), id.end(), 0); }
    ll find(ll a) { return a == id[a] ? a : id[a] = find(id[a]); }

    bool uni(ll a, ll b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (len[a] < len[b]) swap(a, b);
        len[a] += len[b], id[b] = a;
        return true;
    }
};
```

7.5 Segment Tree Over Time (Offline Dynamic Connectivity)

45c

Offline: edges are added/removed over time; DSU can't delete, so each op is active in a time interval [l, r] and stored in a segtree over TIME. A DFS on it applies ops entering a node and rolls back leaving it, so at leaf t the DSU has exactly the ops active at time t. Example: add/remove edges, add value to vertex, query component sum. Edit time_op, apply and leaf for your problem.

Complexity: O(T log T log N)

```
#include "src/structures/dsu_rb.cpp"

struct time_op { ll type, a, b; }; // type 0: edge (a, b); type
1: add b to vertex a

// node covering times [lm, rm]
struct time_node {
    ll lm, rm;
    unique_ptr<time_node> lc, rc;

    vector<time_op> op; // ops active during the WHOLE [lm, rm]

    time_node(ll lm_, ll rm_){
        lm = lm_;
        rm = rm_;
        if (lm != rm) {
            ll mid = (lm + rm) / 2;
            lc = make_unique<time_node>(lm, mid);
            rc = make_unique<time_node>(mid + 1, rm);
        }
    }

    // op x active in [lq, rq]: stored in O(log T) nodes (like a
range update)
    void add_query(ll lq, ll rq, time_op x) {
        if (rq < lm || lq > rm) return;
        if (lq <= lm && rm <= rq) {
            op.push_back(x);
            return;
        }
        lc->add_query(lq, rq, x);
        rc->add_query(lq, rq, x);
    }
};

struct dyn_conn {
    dsu_rb d;

    vll sum, qv, ans; // qv[t] = vertex asked at time t (-1 if none)

    dyn_conn(vll& a, ll T) : d(sz(a)), sum(a), qv(T, -1), ans(T,
-1) {}

    // every change must be undoable: d.save(x) BEFORE changing
extra data
    void apply(time_op& o) {
        if (o.type == 0) {
            ll ra = d.find(o.a), rb = d.find(o.b);
            if (!d.unite(ra, rb)) return;
            ll r = d.find(ra), other = r == ra ? rb : ra; // new
 root / absorbed
            d.save(sum[r]), sum[r] += sum[other];
        } else {
            ll r = d.find(o.a);
            d.save(sum[r]), sum[r] += o.b;
        }
    }

    // here the DSU has exactly the ops active at time t
    void leaf(ll t) {
        if (qv[t] != -1) ans[t] = sum[d.find(qv[t])];
    }

    void dfs(time_node& node) {
        ll t = d.time();
        for (time_op& o : node.op) apply(o); // valid for every
leaf below
        if (node.lm == node.rm) leaf(node.lm);
        else dfs(*node.lc), dfs(*node.rc);
        d.rollback(t); // sibling subtree must not see these ops
    }
};
```

7.6 Fenwick Tree (Binary Indexed Tree)

953

Update pontual e query de prefixo/intervalo em O(log n). Interface 0-based, internamente 1-based. T e um grupo abeliano (mesmo Node da BitRange: T() identidade, a + b, a - b). Se so usar pref, basta monoide comutativo. upper_bound so faz sentido se T tem ordem e todo elemento e >= identidade (ex: somas de valores nao negativos), pois precisa de prefixos monotonos.

Complexity: O(log n) por op, O(n) build

```
template <class T = ll>
struct Bit {
    ll n;
    vector<T> bit;
    Bit(ll _n = 0) : n(_n), bit(n + 1) {}
    Bit(vector<T>& v) : Bit(sz(v)) {
        forn(i,1,n+1) {
            bit[i] = bit[i] + v[i - 1];
            ll j = i + (i & -i);
            if (j <= n) bit[j] = bit[j] + bit[i];
        }
    }
    void update(ll i, const T& x) {  // soma x na posicao i
        for (i++; i <= n; i += i & -i) bit[i] = bit[i] + x;
    }
    T pref(ll i) {  // combina [0, i-1]
        T r{};
        for (; i; i -= i & -i) r = r + bit[i];
        return r;
    }
    T query(ll l, ll r) { return pref(r + 1) - pref(l); }  //
combina [l, r]
```

```
    // menor p (0-based) com pref(p + 1) > x, ou n se nao existir
    // = quantos elementos do comeco cabem com soma <= x
8c1     ll upper_bound(T x) {
62d         ll p = 0;
e63         for (ll k = n ? __lg(n) : -1; k >= 0; k--)
433             if (p + (1LL << k) <= n && bit[p + (1LL << k)] <= x)
2fb                 x = x - bit[p += 1LL << k];
74e         return p;
083     }
676 };
```

7.7 Custom Hash for hash table 634

Provides 64-bit hashers for integers and pairs to use with __gnu_pbds::gp_hash_table.

Complexity: O(1) esperado por operacao, O(n) memoria

```
c4d #include <bits/extc++.h>

// for ll
75f struct chash {
5d6     const uint64_t C = ll(4e18 * acos(0)) | 71;
2cf     ll operator()(ll x) const { return __builtin_bswap64(x*C); }
cdd };

// for pll
75f struct chash {
5d6     const uint64_t C = ll(4e18 * acos(0)) | 71;
fde     ll h(ll x) const { return __builtin_bswap64(x * C); }
344     ll operator()(const pll& p) const { return h(p.first ^ h(p.
second)); }
c63 };

b6a __gnu_pbds::gp_hash_table<ll, ll, chash> h({},{},{},{},{1<<16});
4ca
__gnu_pbds::gp_hash_table<pll, ll, chash> h({},{},{},{},{1<<16});
```

7.8 Mo's Algorithm (Offline Range Queries) 1b2

Responde queries offline em intervalos [l, r] fechados e 0-indexados, ordenando-as (Hilbert ou blocos) para mover pouco os ponteiros. Preencha o estado do problema, insert(p), erase(p) e ans.

Complexity: O((N + Q) sqrt N), O(N + Q)

```
0df struct Mo {
873     ll n, ans = 0;
    // estado do problema aqui

368     Mo(ll n) : n(n) {}

e03     void insert(ll p) {
9d4     }

ec9     void erase(ll p) {
1c8     }

4b3     ll hilbert(ll x, ll y, ll N) {
b72         ll d = 0;
485         for (ll s = N / 2; s > 0; s /= 2) {
6d1             ll rx = (x & s) > 0, ry = (y & s) > 0;
e94             d += s * s * ((3 * rx) ^ ry);
d2e             if (ry == 0) {
5aa                 if (rx == 1) x = N - 1 - x, y = N - 1 - y;
9dd                 swap(x, y);
e2d             }
54d         }
be2         return d;
4b9     }

d1c     vll run(vector<pll>& q, bool use_hilbert = true) {
12a         ll m = sz(q);
17f         vll ord(m), ret(m);
be8         iota(ord.begin(), ord.end(), 0);
67a         if (use_hilbert) {
da0             ll N = 1ll << (__lg(max(n, 1ll)) + 1);
e3f             vll h(m);
8af             forn(i, 0, m) h[i] = hilbert(q[i].first, q[i].second,
 N);
e60             sort(ord.begin(), ord.end(), [&](ll l, ll r) {
return h[l] < h[r]; });
217         }
9d8         if (!use_hilbert) {
2ef             ll SQ = max(1ll, (ll) sqrt(n));
0a3             sort(ord.begin(), ord.end(), [&](ll l, ll r) {
9c9                 if (q[l].first / SQ != q[r].first / SQ) return q
[l].first < q[r].first;
0db                 if ((q[l].first / SQ) % 2) return q[l].second >
q[r].second;
a66                 return q[l].second < q[r].second;
a1d             });
6ab         }
f09         ll l = 0, r = -1;
f99         for (ll i : ord) {
0e7             auto [ql, qr] = q[i];
026             while (r < qr) insert(++r);
232             while (l > ql) insert(--l);
75e             while (l < ql) erase(l++);
fe8             while (r > qr) erase(r--);
381             ret[i] = ans;
701         }
edf         return ret;
8cd     }
1b2 };
```

7.9 Order-Statistic Tree (PBDS) 9b4

Set ordenado com acesso por posicao. s.find_by_order(k): iterador pro k-esimo (0-indexado); end() se k >= sz(s). s.order_of_key(x): quantidade de elementos < x. Para multiset, use ordered_set<pll> com {valor, id unico}.

Complexity: O(log N) per op, O(N)

```
774 #include <ext/pb_ds/assoc_container.hpp>
30f #include <ext/pb_ds/tree_policy.hpp>

0d7 using namespace __gnu_pbds;

7af template<class T> using ordered_set = tree<T, null_type, less<T>,
 rb_tree_tag, tree_order_statistics_node_update>;
```

7.10 Persistent Segment Tree 641

Point updates and range queries across versions. 0-indexed, query on [a, b]. Version 0 is the initial array. set(a, val, t) creates a new version from version t and returns its id. q must be >= the number of set() calls.

Complexity: O(log n) per query/update, O(n + q log n) memory

```
bf2 struct Node {
348     ll val;
f88     ll l, r;
441 };

a49 struct perseg {
4ab     ll n, cnt = 0;
f0a     vector<Node> nd;
d31     vll rt;

cdf     perseg(vll& v, ll q) : n(sz(v)) {
596         nd.resize(2*n + (__lg(n)+2)*q);
72b         rt.reserve(q + 2);
d55         rt.push_back(cnt++);
74d         build(nd.data(), cnt, rt[0], 0, n-1, v);
e69     }
625
    static ll build(Node* d, ll& cnt, ll p, ll l, ll r, vll& v) {
4ae         if (l == r) return d[p].val = v[l];
92f         d[p].l = cnt++; d[p].r = cnt++;
3e3         ll m = (l+r)/2;
6da         return d[p].val = build(d, cnt, d[p].l, l, m, v) + build
(d, cnt, d[p].r, m+1, r, v);
026     }
4c5     ll query(ll a, ll b, ll t) {
d46         return query(nd.data(), a, b, rt[t], 0, n-1);
41a     }
468     static ll query(Node* d, ll a, ll b, ll p, ll l, ll r) {
d5e         if (b < l || r < a) return 0;
1aa         if (a <= l && r <= b) return d[p].val;
3e3         ll m = (l+r)/2;
ebc         return query(d, a, b, d[p].l, l, m) + query(d, a, b, d[p
].r, m+1, r);
3ee     }
9b9     ll set(ll a, ll val, ll t) {
106         ll nr = cnt++;
dda         set(nd.data(), cnt, a, val, rt[t], nr, 0, n-1);
4fd         rt.push_back(nr);
cfb         return (ll)rt.size()-1;
91a     }
c4d     static ll set(Node* d, ll& cnt, ll a, ll val, ll lp, ll p,
ll l, ll r) {
8f3         if (l == r) return d[p].val = val;
3e3         ll m = (l+r)/2;
330         if (a <= m) {
198             d[p].l = cnt++; d[p].r = d[lp].r;
f2a             return d[p].val = set(d, cnt, a, val, d[lp].l, d[p].
l, l, m) + d[d[p].r].val;
5c0         }
578         d[p].l = d[lp].l; d[p].r = cnt++;
bda         return d[p].val = d[d[p].l].val + set(d, cnt, a, val, d[
lp].r, d[p].r, m+1, r);
040     }
443 };
```

7.11 Segment Tree (Range Query + Range Update) ad5

Seg lazy generica, intervalos [i, j] fechados e 0-indexados. Node::operator combina dois nos. Update::operator() aplica o update num no. Update::operator+ compoe: (a + b) = aplica a, depois b. Exemplo: soma com add/set em intervalo (folhas devem ter cnt = 1).*

Complexity: O(log N) per op, O(N) to build

```
bf2 struct Node {
946     ll sum = 0, cnt = 0;
298     Node operator*(const Node &o) const { return {sum + o.sum,
cnt + o.cnt}; }
d84 };
```

```
struct Update {
    ll add = 0;
    optional<ll> set;

    Node operator()(const Node &n) const {
        ll res = set.has_value() ? *set * n.cnt : n.sum;
        return {res + add * n.cnt, n.cnt};
    }

    Update operator+(const Update &o) const {
        Update res = *this;
        if (o.set.has_value()) {
            res.set = o.set;
            res.add = 0;
        }
        if (o.add != 0) {
            if (res.set.has_value()) *res.set += o.add;
            else res.add += o.add;
        }
        return res;
    }
};

template<typename T, typename U> struct segtree {
  ll s, h;

  T id;
  vector<T> val;

  U noop;
  vector<bool> dirty;
  vector<U> prop;

  segtree(ll ts, T tid = T(), U tnoop = U()) {
    id = tid, noop = tnoop;
    for (s = 1, h = 1; s < ts; ) s *= 2, h++;

    val.assign(2*s, id);
    dirty.assign(2*s, false);
    prop.assign(2*s, noop);
  }

  void set_leaves(vector<T> &lvs) {
    copy(lvs.begin(), lvs.end(), val.begin() + s);

    for (ll i = s - 1; i > 0; i--) val[i] = val[2 * i] * val[2 * i + 1];
    dirty.assign(2*s, false);
    prop.assign(2*s, noop);
  }

  void apply(ll i, U &upd) {
    val[i] = upd(val[i]);
    if(i < s) {
      prop[i] = prop[i] + upd;
      dirty[i] = true;
    }
  }

  void pull(ll i) {
    for (ll l = i/2; l; l /= 2) {
      T comb = val[2*l] * val[2*l+1];
      val[l] = prop[l](comb);
    }
  }

  void push(ll i) {
    for (ll th = h - 1; th > 0; th--) {
      ll l = i >> th;

      if (dirty[l]) {
        apply(2*l, prop[l]);
        apply(2*l+1, prop[l]);

        prop[l] = noop;
        dirty[l] = false;
      }
    }
  }

  void update(ll i, ll j, U upd) {
    i += s, j += s;
    push(i), push(j);

    for (ll l = i, r = j; l <= r; l /= 2, r /= 2) {
      if((l&1) == 1) apply(l++, upd);
      if((r&1) == 0) apply(r--, upd);
    }

    pull(i), pull(j);
  }

  T query(ll i, ll j){
    i += s, j += s;
    push(i), push(j);

    T rl = id, rr = id;
    for(; i <= j; i /= 2, j /= 2){
      if((i&1) == 1) rl = rl * val[i++];
      if((j&1) == 0) rr = val[j--] * rr;
    }
    return rl * rr;
  }
};
```

7.12 Segment Tree (Range Query + Point Update) a25

segtree<node> S(n); set_leaves(v), update(i, v), query(l, r) on [l, r]. node::comb is static and only needs to be associative (order is preserved). max_right(l, x): smallest r >= l with pred(comb(a[l..r]), x), or n if none. pred is a member of segtree (edit it); default: sum > x (values >= 0). ex: first prefix with sum > x -> max_right(0, x) Size is rounded up to a power of 2 (max_right needs it).

Complexity: O(log N) per op, O(N) mem

```
struct node {
    ll val = 0;

    static node comb(const node& a, const node& b) {
        return {a.val + b.val};
    }
};

template<typename T> struct segtree {
  ll n, s;
  T neutral;
  vector<T> tree;

  segtree(ll _n, T _neutral = T()) {
    n = _n, neutral = _neutral;
    for (s = 1; s < n; ) s *= 2;
    tree.assign(2*s, neutral);
  }

  void set_leaves(vector<T> &leaves) {
    copy(leaves.begin(), leaves.end(), tree.begin() + s);

    for (ll i = s - 1; i > 0; i--)
      tree[i] = T::comb(tree[2 * i], tree[2 * i + 1]);
  }

  // for SET segtree
  // for (i += s; i; i /= 2)
  //    tree[i].erase(tree[i].find(old)), tree[i].insert(new);
  void update(ll i, T v) {
    i += s;
    tree[i] = v;
    while (i > 1) {
      i /= 2;
      tree[i] = T::comb(tree[2 * i], tree[2 * i + 1]);
    }
  }
  // for SET segtree, same loop (ex: smallest >= k)
  //    if((i&1) == 1) ans = min(ans, *tree[i++].lower_bound(k));
  //    if((j&1) == 0) ans = min(ans, *tree[j--].lower_bound(k));
  T query(ll i, ll j) {
    T rl = neutral, rr = neutral;
    for(i += s, j += s; i <= j; i /= 2, j /= 2){
      if((i&1) == 1) rl = T::comb(rl, tree[i++]);
      if((j&1) == 0) rr = T::comb(tree[j--], rr);
    }
    return T::comb(rl, rr);
  }

  bool pred(const T& a, ll x) { return a.val > x; }

  ll max_right(ll l, ll x) {
    if (l == n) return n;
    T acc = neutral;
    ll v = l + s;
    do {
      while ((v&1) == 0) v /= 2; // biggest block from here
      if (pred(T::comb(acc, tree[v]), x)) { // descend in v
        while (v < s) {
          v *= 2;
          if (!pred(T::comb(acc, tree[v]), x))
            acc = T::comb(acc, tree[v++]);
        }
        return v - s;
      }
      acc = T::comb(acc, tree[v++]);
    } while (v & (v-1)); // past the end
    return n;
  }
};
```

7.13 Sparse Table (Idempotent Range Query) 349

Preprocesses static array to answer idempotent range queries (e.g., min/max) in O(1) after O(N log N) build.

Complexity: O(N log N) build, O(1) query; O(N log N) space

```
991 struct sparse {
815     vector<vll> m;
78e     sparse(vll& v) : m(1, v) {
dce         ll n = sz(v);
d24         for (ll j = 1; (1LL << j) <= n; j++) {
1bf             m.emplace_back(n - (1LL << j) + 1);
425             forn(i, 0, sz(m[j]))
d20                 m[j][i] = min(m[j-1][i], m[j-1][i + (1LL << (j-1)
)]);
45c         }
9d2     }
4de     ll query(ll a, ll b) {  // min de [a, b], exige a <= b
e21         ll j = __lg(b - a + 1);
57a         return min(m[j][a], m[j][b - (1LL << j) + 1]);
6a4     }
349 };
```

7.14 Implicit Treap (Sequence Treap) 2a5

Maintains a sequence with split and merge operations using randomized priorities and subtree sizes.

Complexity: O(log N) expected per op, O(N)

```
125 struct Treap{
348     ll val;
0ce     ll prio, size;
330     vector<Treap*> kids;
b02     Treap(ll c): val(c), prio(rand()), size(1),
680         kids({NULL,NULL}){};
494 };

464 ll size(Treap *me){return me ? me->size : 0;}
e86 void rsz(Treap* me){me -> size =
24d     1 + size(me->kids[0]) + size(me->kids[1]);}

e8f vector<Treap*> split(Treap *me, ll idx){
878     if(!me) return {NULL,NULL};
032     vector<Treap*> out;

52a     if(size(me->kids[0]) < idx){
e1c         auto aux = split(me->kids[1],
312             idx - size(me->kids[0]) -1);
409         me->kids[1] = aux[0];
b14         rsz(me);
abb         out = {me, aux[1]};
aaa     }else{
c8a         auto aux = split(me->kids[0], idx);
c89         me->kids[0] = aux[1];
b14         rsz(me);
3cb         out = {aux[0], me};
d61     }
fe8     return out;
e7d }

b85 Treap* merge(Treap *left, Treap *right){
c10     if(left == NULL) return right;
096     if(right == NULL) return left;

671     Treap* out;
38d     if(left->prio < right->prio){
d90         left->kids[1] = merge(left->kids[1], right);
122         rsz(left);
d7a         out = left;
bbb     }else{
cea         right->kids[0] = merge(left, right->kids[0]);
e85         rsz(right);
015         out = right;
2f1     }
fe8     return out;
499 }
```

1 Counting Problems

1.1 Burnside's Lemma

Let G be a group that acts on a set X. The Burnside Lemma states that the number of distinct orbits is equal to the average number of points fixed by an element of G.

$$T = \frac{1}{|G|} \sum_{g \in G} |\texttt{fix}(g)|$$

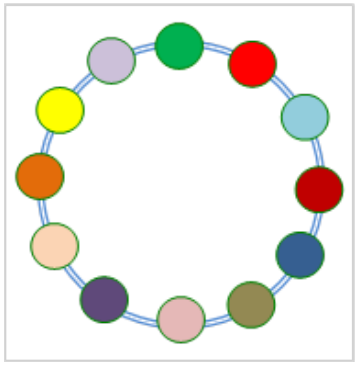
Where a orbit $\texttt{orb}(x)$ is defined as

$$\texttt{orb}(x) = \{y \in X : \exists g \in G \; gx = y\}$$

and $\texttt{fix}(g)$ is the set of elements in X fixed by g

$$\texttt{fix}(g) = \{x \in X : gx = x\}$$

Example: With k distinct types of beads how many distinct necklaces of size n can be made? Considering that two necklaces are equal if the rotation of one gives the other.



$$T = \frac{1}{n} \sum_{i=0}^{n-1} k^{gcd(i,n)}$$

1.2 Catalan Numbers

1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440, 9694845, 35357670, 129644790, 477638700, 1767263190, 6564120420, 24466267020, 91482563640, 343059613650, 1289904147324, 4861946401452, 18367353072152, 69533550916004, 263747951750360, 1002242216651368.

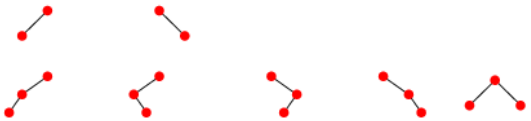
$$C_n = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{(n+1)!n!} = \prod_{k=2}^{n} \frac{n+k}{k}, \quad n \geq 0$$

Applications:

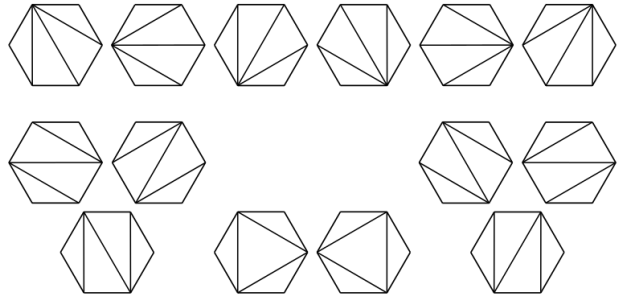
- C_n counts the number of expressions containing n pairs of parentheses which are correctly matched.

  ((())) ()(()) ()()() (())() ...

- Successive applications of a binary operator can be represented in terms of a full binary tree. (A rooted binary tree is full if every vertex has either two children or no children.) It follows that C_n is the number of full binary trees with n + 1 leaves:



- C_n is the number of different ways a convex polygon with n + 2 sides can be cut into triangles by connecting vertices with straight lines (a form of Polygon triangulation). The following hexagons illustrate the case n = 4:



2 Graphs

2.1 Sum of Subtrees of a Tree

For a rooted tree T with n vertices, let $sz(v)$ be the size of the subtree of v. Then the following holds:

$$\sum_{v \in V} \left[\text{sz}(v) + \sum_{u \text{ child of } v} \text{sz}(u)(\text{sz}(v) - \text{sz}(u)) \right] = n^2$$

3 Number Theory

3.1 Chicken McNugget Theorem

Given two **coprime** numbers n, m, the largest number that cannot be written as a linear combination of them is $nm - n - m$.

- There are $\frac{(n-1)(m-1)}{2}$ non-negative integers that cannot be written as a linear combination of n and m;
- For each pair $(k, nm - n - m - k)$, for $k \geq 0$, exactly one can be written.

3.2 Identities

$$\sum_{d|n} \varphi(d) = n$$

$$\sum_{\substack{i<n \\ \gcd(i,n)=1}} i = n \cdot \frac{\varphi(n)}{2}$$

$$|\{(x,y) : 1 \leq x, y \leq n,\ \gcd(x,y) = 1\}| = \sum_{d=1}^{n} \mu(d) \left\lfloor \frac{n}{d} \right\rfloor^2$$

$$\sum_{x=1}^{n}\sum_{y=1}^{n} \gcd(x,y) = \sum_{k=1}^{n} k \sum_{k|l}^{n} \left\lfloor \frac{n}{l} \right\rfloor^2 \mu\left(\frac{l}{k}\right)$$

$$\sum_{x=1}^{n}\sum_{y=x}^{n} \gcd(x,y) = \sum_{g=1}^{n}\sum_{g|d}^{n} g \cdot \varphi\left(\frac{d}{g}\right)$$

$$\sum_{x=1}^{n}\sum_{y=1}^{n} \operatorname{lcm}(x,y) = \sum_{d=1}^{n} d\,\mu(d) \sum_{d|l}^{n} l \binom{\lfloor \frac{n}{l} \rfloor + 1}{2}^2$$

$$\sum_{x=1}^{n}\sum_{y=x+1}^{n} \operatorname{lcm}(x,y) = \sum_{g=1}^{n}\sum_{g|d}^{n} d \cdot \varphi\left(\frac{d}{g}\right) \cdot \frac{d}{g} \cdot \frac{1}{2}$$

$$\sum_{x\in A}\sum_{y\in A} \gcd(x,y) = \sum_{t=1}^{n} \left(\sum_{l|t} \frac{t}{l}\mu(l)\right) \left(\sum_{t|a} \text{freq}[a]\right)^2$$

$$\sum_{x\in A}\sum_{y\in A} \operatorname{lcm}(x,y) = \sum_{t=1}^{n} \left(\sum_{l|t} \frac{l}{t}\mu(l)\right) \left(\sum_{a\in A,\ t|a} a\right)^2$$

3.3 Large Prime Gaps

For numbers until 10^9 the largest gap is 400.
For numbers until 10^{18} the largest gap is 1500.

3.4 Lucas's Theorem

$$\binom{n}{m} \equiv \prod_{i=0}^{k} \binom{n_i}{m_i} \pmod{p}$$

For p prime. n_i and m_i are the coefficients of the representations of n and m in base p.

3.5 Mobius Inversion

If $g(n) = \sum_{d|n} f(d)$, then $f(n) = \sum_{d|n} g(d)\mu(\frac{n}{d})$.
A more useful definition is: $\sum_{d|n} \mu(d) = [n = 1]$
Example, sum of LCM:

$$\begin{aligned}
\sum_{i=1}^{n}\sum_{j=1}^{n} \operatorname{lcm}(i,j) &= \sum_{k=1}^{n}\sum_{i=1}^{n}\sum_{j=1}^{n} [\gcd(i,j) = k]\frac{ij}{k} \\
&= \sum_{k=1}^{n}\sum_{a=1}^{\lfloor \frac{n}{k} \rfloor}\sum_{b=1}^{\lfloor \frac{n}{k} \rfloor} [\gcd(a,b) = 1] abk \\
&= \sum_{k=1}^{n} k \sum_{a=1}^{\lfloor \frac{n}{k} \rfloor} a \sum_{b=1}^{\lfloor \frac{n}{k} \rfloor} b \sum_{d=1}^{\lfloor \frac{n}{k} \rfloor} [d|a]\,[d|b]\,\mu(d) \\
&= \sum_{k=1}^{n} k \sum_{d=1}^{\lfloor \frac{n}{k} \rfloor} \mu(d) \left(\sum_{a=1}^{\lfloor \frac{n}{k} \rfloor} [d|a]\,a\right) \left(\sum_{b=1}^{\lfloor \frac{n}{k} \rfloor} [d|b]\,b\right) \\
&= \sum_{k=1}^{n} k \sum_{d=1}^{\lfloor \frac{n}{k} \rfloor} \mu(d) \left(\sum_{p=1}^{\lfloor \frac{n}{kd} \rfloor} p\right) \left(\sum_{q=1}^{\lfloor \frac{n}{kd} \rfloor} q\right)
\end{aligned}$$

3.6 Number of Divisors

n	6	60	360	5040	55440	720720	4324320	21621600
$d(n)$	4	12	24	60	120	240	384	576

n	367567200	6983776800	13967553600	321253732800
$d(n)$	1152	2304	2688	5376

184010559381256608000 $\approx$ 2e18 is highly composite with 184320 divisors.
For numbers up to 10^{88}, $d(n) < 3.6\sqrt[3]{n}$.

3.7 Prime Counting Function - $\pi(x)$

The prime counting function is asymptotic to $\frac{x}{\log x}$, by the prime number theorem.

x	10	10^2	10^3	10^4	10^5	10^6	10^7	10^8
$\pi(x)$	4	25	168	1 229	9 592	78 498	664 579	5 761 455

3.8 Some Primes

999999937 1000000007 1000000009 1000000021 1000000033
$10^{18} - 11$ $10^{18} + 3$ $2305843009213693951 = 2^{61} - 1$